

A Theoretical Foundation and Practical Demonstration of Integrating MDO, MBSE and the Digital Thread

Christopher A. Lupp*, Jason Y. Kao†, Neal Novotny‡, and Timothy Wontor§
Air Force Research Laboratory, Wright-Patterson AFB, Ohio, 45433

Alexander Xu¶, James Singleton||, and David A. Sandler**
University of Dayton Research Institute, Dayton, Ohio, 45469, USA

Multidisciplinary Analysis, Design, and Optimization (MDO) has become an important part of aerospace vehicle development. Recently, a drive to increase agility during design has led to initiatives by multiple organizations to integrate Model-Based Systems Engineering (MBSE) and MDO. The RQV Effectiveness Assessment of Concepts and Technologies (REACT) project has pursued this integration using a centralized data strategy with clearly defined, open interfaces to enable the goal of closer integration. **The paper proposes a new theoretical foundation to describe the interaction of data, MBSE, and MDO and their relationship to the Digital Thread. It details the development of an authoritative source of truth, Nucleus, to serve as a common MBSE-MDO database. The fundamental and generalized ontology concepts, that make this approach applicable to a large number of design problems, are presented. Traceability of data is assured by version control.** The paper details practical demonstrations of design tools, including high-fidelity coupled analyses, integrating with the MBSE-MDO database. Finally, ramifications of this approach are discussed, including effects to adopting organizations.

Nomenclature

K_{MBSE}	MBSE model key	c_{wing}	wing chord, m
K_{MDO}	MDO model key	m_f	fuel burn mass, kg
$\mathcal{M}$	MBSE-MDO map	t_i	wing box thickness, m
T_0	engine static thrust, N	Π	over-all pressure ratio
ζ_t	turbine entry temperature, K	α_b	body angle of attack, $^\circ$
V_{fuel}	fuel volume, m^3	μ	engine bypass ratio
V_{wb}	wing box volume, m^3	ν	model variable value
b_{body}	body section span, m	σ	wing box stress, Pa
b_{wing}	wing span, m	σ_{yield}	yield stress, Pa
c_{body}	body section chord, m		

*Research Aerospace Engineer, Aerospace Systems Directorate, christopher.lupp@us.af.mil, AIAA Associate Fellow

†Research Aerospace Engineer, Aerospace Systems Directorate

‡Research Aerospace Engineer, Aerospace Systems Directorate, AIAA Young Professional Member

§Research Aerospace Engineer, Aerospace Systems Directorate

¶Research Computer Scientist

|| Research Computer Scientist

**Aerospace Engineer, AIAA Young Professional Member

I. Introduction

ENGINEERING design, deployment, and sustainment of complex systems involves multiple distinct phases and a large variety of disciplines and data sources. The entirety of this process is known as Product Lifecycle Management (PLM), and encompasses the system from conceptualization through its entire product life. While tools and processes exist for PLM within the detailed design, deployment, sustainment, and disposal phases, the research and early design phase(s), have different requirements that have remained unmet. However, a desire to more closely connect stakeholders, system engineers, and multi-disciplinary design optimization (MDO) engineers; a need to decrease design phase duration; and a drive to increase traceability for early design has recently driven multiple independent research efforts to integrate with model-based systems engineering (MBSE) and adopt Digital Engineering practices such as integration with a Digital Thread.

Digital Engineering practices and Digital Thread, in particular, have become central to tightly integrate and accelerate various design phases. The exact origin of the term “Digital Thread” is difficult to trace, as the term appears to have been first used outside of formal literature. While Wu [1] claims that the term Digital Thread originated in a magazine article in 2010, Hamstra [2] notes that the term was coined by the Air Force Research Laboratory (AFRL) and Lockheed Martin during the nascent F-35 program of the early 2000s. Regardless of its exact origin, the Digital Thread has been the subject of a plethora of research across a variety of fields. A Digital Thread connects data that has been created and collected during the product’s lifecycle. Correct application of a Digital Thread promises to accelerate design phases and provide insights into decision making throughout the entire product’s development and life.

Numerous articles by Kraft [3–9] at AFRL, primarily focus on the role of the Digital Thread on vehicle or system acquisition and lifecycle management. Also within AFRL, Kobryn et al. [10] introduced multiple Digital Thread concepts to improve the acquisition and management of aerospace systems throughout their lifecycle. While they briefly discussed design, their work centered on later design phases such as detailed design, manufacturing and sustainment. Additionally, their work was limited to conceptual findings rather than expanding these concepts to tools and practical implementation. Later, Gharbi et al. [11] proposed a Digital Thread approach for the detailed design of an aircraft rib. Singh and Willcox [12] presented a Digital Thread approach within a single design stage of an aircraft rib. While they proposed a mathematical representation of the Digital Thread, implementation details such as data architecture and locality remained unaddressed.

While data and its traceability is important, data must first be created (and be utilized) by analysis or design processes. As one of these processes, MDO has become a valuable tool in the design of complex systems such as aircraft and aerospace systems. A precursor to integrating MDO to MBSE and Digital Engineering practices is the enabling of large-scale, geographically-distributed MDO, as many problems complex design problems occur across organizational boundaries (e.g., airframer and engine manufacturer).

AFRL’s Multidisciplinary Science and Technology Center (MSTC)—a part of AFRL’s Air Vehicle Division (AFRL/RQV)—has a long heritage of developing and transitioning computationally distributed MDO practices. The SORCER framework [13, 14] was developed and used for in-house MDO studies [15–24]. SORCER follows a service-based and computationally distributed approach to calling individual disciplines. Later, Snyder [25] developed a computationally distributed discipline protocol called Hermes using zeroMQ. This work also followed a server/client methodology and lent itself to geographically distributed computing. More recently, Lupp and Xu [26] proposed a standardized interface for multi-disciplinary analysis and design optimization (MADO) disciplines, Philote, based on the widely used gRPC [27] protocol. Philote is actively used at AFRL today and is being refined by the AIAA MDO Technical Committee Standards Working Group. The working group aims to mature a reliable standard interface that is platform and framework agnostic. Like SORCER and Hermes before it, Philote enables computationally/geographically distributed workflows, extending it to gradient-based optimization. Finally, transition of computationally distributed optimization concepts was achieved with AFRL’s OPTIMUS (with prime contractor Boeing) [28] and EXPEDITE (with prime contractor Lockheed Martin) [29, 30] programs demonstrating computationally distributed MDO processes between prime and a variety of sub-contractors at relevant industrial scale.

Likewise, the European AGILE 3.0 project [31–33] focused on developing and demonstrating tools and processes for large scale, distributed MDO in an industrial setting. While AGILE 3.0 focused on distributed MDO [31], their computationally distributed architecture laid the groundwork for digital engineering (DE) workflows. Moreover, they conceptualized and implemented centralized data schemata [32, 33] that could be considered a predecessor of MBSE integration and creating traceability via a Digital Thread.

Another MDO effort that started to bridge MDO and MBSE in 2017 was VOLTA [34], developed by ESTECO. VOLTA is a Simulation Process and Data Management (SPDM) solution that allows multiple collaborators to work together in a central data location to create and run MDO workflows. With the release of a Cameo MBSE plugin in

2025, MBSE SysML diagrams can be connected to MDO workflows through the SPDM, providing a commercial option for MBSE/MDO integration. Similarly, SBE Vision provides another commercial software for managing engineering data across a variety of tools and workflows.

In 2019, AFRL/RQV conceived the RQV Effectiveness Assessment of Concepts and Technologies (REACT) project to improve in-house design processes to aid technology investment decisions and more closely engage customers/stakeholders. Bridging MADO/MDO and MBSE was identified as key to reaching these goals. As the project matured, discovery and integration of Digital Engineering practices into in-house design processes and transitioning tools and methods to the wider aerospace and MDO fields became a high priority.

Early in the project, project members conducted a survey of the MDO and MBSE fields and tools available, identifying vital gaps that needed to be addressed:

- Need for a central, vendor-neutral authoritative source of truth (ASOT)
- The ASOT needs to be built on modern database technologies that allowed horizontal scaling at little notice
- The ASOT needs to be capable of supporting data and artifacts ranging from low to high fidelity physics
- Need for a standardized MBSE architecture/a Government Reference Architecture (GRA) for air vehicle applications
- Support of both MDO and MBSE workflows, independently

To address these needs, the REACT team adopted and fine-tuned a MBSE government reference architectures (GRA), devised and developed a database service with a standardized API to serve as the ASOT; implemented a Digital Engineering framework for MBSE and MDO; and demonstrated integrated MBSE and MDO workflows. A key element of the REACT approach is the central database (presented in this work), which serves as a vendor-neutral ASOT and creating open/standardized interfaces for MADO/MDO and MBSE to the ASOT. This approach allows interaction via system engineering tools (e.g., Cameo), analyses, or MDO frameworks without vendor lock concerns. As a primary means of technology transition, AFRL/RQV disseminated its findings through direct engagement with industry and other government researchers.

To create a common MBSE reference architecture for air vehicle design, a GRA based on MIL-STD-881E [35] was adopted from the Air Force Digital Transformation Office. This preliminary GRA provided a broad work breakdown structure (WBS) for air vehicle concepts. In 2021, the AFRL/RQV REACT team implemented this preliminary WBS in an MBSE environment, and added finer details for logical and physical descriptions for each of the relevant components originally listed. This developmental process was coordinated with diverse stakeholders from across the Aerospace Systems Directorate (AFRL/RQ) and AFRL.

In 2022 the initial database and API were released internally, an MBSE/Cameo interface was released internally, and an integrated MBSE/aircraft design workflow was demonstrated (the latter in collaboration with Naval Air System Command (NAVAIR) and Computational Research and Engineering Acquisition Tools and Environments - Air Vehicles (CREATE-AV)). The demonstration featured a systems engineer modifying vehicle parameters, synchronizing it with the database (ASOT), before a remotely located conceptual design engineer evaluated the synchronized configuration using CREATE-AV Aircraft Design, Analysis, Performance, and Trade-space (ADAPT) [36, 37]. The results of this analysis were again synchronized with the ASOT via an open API, allowing the systems engineer to evaluate the effect on system requirements. Further demonstrations were conducted in 2024 (integrated REACT framework and automated triggering of MDO optimizations through the ASOT) and 2025 (synchronizing high-fidelity data to/from the ASOT).

As is typical for AFRL and the defense community, tool and method transition to government and industrial partners is a key objective of the REACT project, often taking precedence over formal, published literature. As previously discussed, a collaboration with NAVAIR and CREATE-AV yielded an integrated MBSE/aircraft design demonstration using CREATE-AV ADAPT. In addition to collaborative technical work, the REACT project engaged with government and industry partners through technical interchange meeting (TIM)s. Technical exchanges within the government started with NAVAIR in early 2022 and National Aeronautics and Space Administration (NASA) in Fall 2022, which continue through the present. REACT's contribution to these interchange meetings was to present the project's detailed architecture (GRA, framework design, database architecture, MBSE interface architecture and design, etc.), as well as detailed demonstrations of developed tools and frameworks. Starting in 2023, these interchange meetings were expanded to include AFRL strategic partners (airframers, tool vendors, more government entities). Also in 2023, AFRL/RQV presented their demonstrations and findings in a session dedicated to REACT and its leveraged projects at the AVTech Symposium [38], attended by multiple government agencies and all major American airframers. As a result, the REACT framework and approach are built on AFRL's long legacy of work in Digital Engineering and the Digital Thread and have been widely disseminated within the United States, influencing other organizations' MDO/MBSE and Digital Engineering strategies.

Independent of and in parallel to AFRL’s REACT progress, the European Union (EU) funded the AGILE 4.0 project, led by the DLR. AGILE 4.0 “targets new methods and technologies to improve, streamline and accelerate the development of complex systems, addressing all the main pillars of the aeronautical supply-chain: design, production, certification and manufacturing” [39]. It built upon the distributed MDO tools developed in AGILE 3.0 and also relied on MBSE integration to support its goals. Ciampa et al. [40] presented a road map to the AGILE 4.0 project, outlining the process from stakeholder inputs to engineering studies. Bussemaker et al. [41] developed a method to link MDO and MBSE architecting processes. Boggero et al. [42] presented an overview of the AGILE 4.0 MBSE-MDO framework. They presented a requirements definition process that spanned multiple organizations. The AGILE 4.0 project put more emphasis on the systems architecting phase [43–46] than REACT. By comparison, REACT also focuses on the capturing of requirements to bring stakeholders and designer closer, but focused more on the MADO/MDO aspect of the project and integrating it to MBSE. This is natural, as MSTC is at its heart an MADO/MDO organization, while AGILE 4.0 was run by the Institute of System Architectures at the DLR. As a result AGILE 4.0 took a more MBSE-centric approach than REACT, resulting in two fundamentally different approaches to the same problem.

The AGILE 4.0 framework was applied in a variety of studies to demonstrate its features and capabilities. In 2021, Cabaleiro et al. [47] used the AGILE framework to conduct technology assessments for multiple candidate technologies on a regional jet. In 2022, Mandorino et al. [48] applied the AGILE 4.0 framework to a regional jet retrofit design process. Fioriti et al. [49] demonstrated a certification-driven design process using the AGILE framework that connected system needs, certification requirements, and the design of a regional jet. Saves et al. [50] investigated the optimization of a distributed electric aircraft using Bayesian Optimization to solve the mixed integer problem by using dimension reduction techniques.

Allison et al. [51] documented an industry-based perspective on implementing model-based engineering (MBE) in 2021, offering insights for phased implementation of MBE. In 2023, Allison et al. [52] expounded on the need for clearly defined ASOT. They asserted that ASOT must be allowed to move from model to model and be able to change in time. In enabling this, they introduced a consistency concept, which allows ASOT to migrate while retaining its authority, for ASOT and methods of enforcement for digital ecosystems.

In 2023, Roohi et al. [53] implemented a Digital Thread framework that connected MBSE and MDO. They created a minimum viable product/proof of concept to illustrate the Digital Thread on the design of a Design, Build, Fly (DBF) small unmanned aerial vehicle (UAV). They used a SQL database to store structured and unstructured data allowing individual components of the digital thread (i.e., MBSE tools) to exchange and manipulate data. However, as a proof of concept, implementation of their presented framework within an enterprise environment would prove challenging. Moreover, in practice a Digital Thread would need to support large numbers of collaborators, authentication, enforcement of data rights, conflict mitigation, etc.

Starting in late 2023, NASA funded the Model-Based Systems Analysis and Engineering (MBSA&E) project, with Boeing as the prime contractor performing the technical activities. Prior and parallel to this, NASA conducted its own in-house MBSA&E exploratory activity, resulting in applications to the Transonic Truss-Braced Wing concept [54]. Early in the MBSA&E project, the REACT team provided technical guidance and details of its own implementations of MBSE-MDO integration, SysML reference architecture implementation, etc. to NASA to aid the project. Boeing had also been a part of prior REACT-industry TIMs.

Carrere et al. [55] presented an overview of the contracted NASA MBSA&E work conducted by Boeing, the University of Michigan, General Electric, and Collins Aerospace, contextualizing their work within the overall goals of NASA’s MBSA&E strategy. They outlined key pillars of the project: the Aircraft Data Hierarchy (ADH) [56], the Standard Evaluator [57], MBSA-MBSE bridge [58], Comprehensive and Narrow Model Sharing [59], and the Aircraft Model Benchmark [60]. Engelbeck et al. [56] developed the ADH which, like REACT’s GRA, is based on MIL-STD-881 [35] and provides a systems engineering reference architecture and file format for aircraft design. Gablonsky et al. [57] developed the Standard Evaluator, which provides a Python library to interface with MDO disciplines via REST. The library provides an interface to OpenMDAO required by the MBSA&E project. Conceptually similar to the REACT database and its MBSE-MDO/aircraft design demonstration, Mokotoff et al. [58] provided an implementation of an API for the ADH that enables the coupling of MBSE and MDO workflows including version control within the NASA MBSA&E framework. Arias et al. [59] presented Boeing’s approach to sharing models and collaborating within MBSA&E workflows. Finally, Wakayama et al. [60] created and provided open models of notional transport aircraft and used them to apply the MBSA&E toolset.

This paper presents the theoretical and practical foundation for integrating MDO and MBSE, and providing the traceability of data, requirements, and design decisions needed for connecting early design phases to the Digital Thread. While the data components of the REACT project are key to its success, the project encompasses a wider set of goals,

many of which are outside the scope of this paper. AFRL’s REACT project work related to data storage, traceability, and interoperability between systems engineering, design, and analysis use-cases, are detailed, including:

- a theoretical foundation of data in MDO and the Digital Thread, as well as a graphical means to capture data and data locality within MDO workflows.
- the creation of a central, vendor- and tool-agnostic, and enterprise-grade database to serve as a government ASOT, storing structured, unstructured data, and metadata. The database was designed to store generalized data, independent of the type of analysis or design being conducted, while providing version control for traceability.
- the definition of an open interface to the database allowing interactions from MDO frameworks, MBSE environments, various analyses (including up to high-fidelity data), etc.
- demonstrations of the database’s utility in connecting MBSE and MDO workflows, traceability of design decisions, and high-fidelity analyses.
- identification of the role and effect of the ASOT on enterprise policies (cybersecurity, data governance, etc.) and collaboration.

To demonstrate the workflows and capabilities that REACT’s approach to data enables, the paper presents four vignettes. They illustrate how data is curated, used with diverse workflows, and exchanged between MBSE and MDO workflows to ensure traceability. These demonstrations span different research and design phases, ranging from a design of experiments (DOE) during early conceptual design using CREATE-AV ADAPT [36, 37], to gradient-based MDO and high-fidelity aerostructural analyses. The latter is particularly noteworthy, as it illustrates the ability to support industry-relevant scales of data and large-scale design problems. Finally, the paper discusses ramifications of REACT’s integrated approach to Digital Engineering on team collaboration, workflows, organizational policies, and culture.

II. Theoretical Foundations of Integrating MADO/MDO and MBSE

Early in the REACT project, integrating MBSE and MDO were identified as key enablers to include stakeholder input and requirements into the design process. MBSE and traceability requirements, combined with potentially large datasets from MDO, posed challenges that needed to be overcome. Central to these challenges were concerns of how and where data needed to be stored and synchronized to create the MBSE-MDO framework. Data architectures and locality are key concepts that drive complexity, based on synchronization needs.

Furthermore, data and data locality can also have significant practical implications including query latency, cybersecurity, etc. Capturing and documenting data, its locations, and which computational boundaries it must transit are imperative. Unfortunately, MDO lacks a representation of data beyond the notion of discipline inputs and outputs that may be coupled. No formal convention exists that allows engineers to represent where a data source is located and what boundaries it must traverse to be received at relevant data sinks.

This section provides a theoretical overview of data architectures, locality, and how they relate to MDO and MBSE. As MDO does not provide sufficient definitions to capture relevant data locations, we extend on previous work to add concept of data blocks, data distribution, and data rights to accompany the computationally, and architecturally distributed concepts introduced by Lupp et al. [61]. We also propose a graphical method to represent the data concepts introduced. Finally, this section details the mapping required to connect MBSE and MDO processes.

A. Data Locality and Architectures

Data locations and its fundamental architecture is paramount whether creating a Digital Engineering framework or a commercial cloud application (e.g., video streaming service). Where data is stored and how complex data interactions are can determine whether an application is feasible or not. This is particularly true for MBSE and MDO, as both fields rely on data and the concept of an authoritative source of truth (ASOT).

Differentiating between data concepts such as schemata and ontologies as well as data architecture and locality is necessary to sufficiently describe data within the MBSE-MDO application. However, because terminology is often used interchangeably—even when they are not—the following provides a brief set of definitions for clarity:

- **data location/locality:** defines where data is logically/physically stored
- **data architecture:** for the purposes of this work, a data architecture describes the data locality, how it connects to data producers and consumers (data sources and sinks), and how these connections behave (e.g., synchronization, data exchange, etc.)
- **ontology:** a formal representation of knowledge as concepts with properties and how they relate to one another within a specific domain.
- **schema:** defines the structure of data (e.g., how a database table is structured)

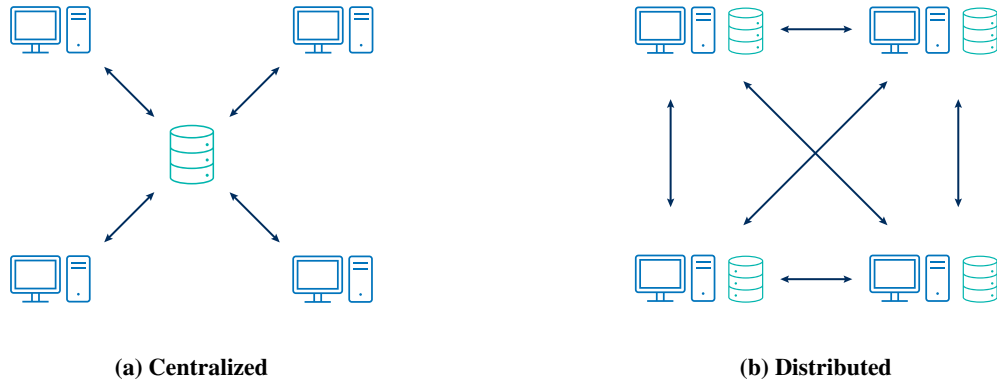


Fig. 1 Centralized and distributed data architectures and their respective required connections.

Reasonable decisions regarding data locality and architecture are paramount when designing a Digital Engineering/MBSE-MDO framework. This is particularly true, as data producers and consumers in such a framework are inherently distributed (not co-located). As a result, two natural expressions of data architectures exist: distributed and centralized data (Figure 1).

A centralized data architecture stores all data/the ASOT in a central location, from which all data producers and consumers can access it. This provides simple data interfaces, as each producer/consumer only needs one direct connection to the ASOT. A common use case for the centralized data architecture may be a specific design phase (e.g., conceptual design). In that scenario, all data required during conceptual design is stored in central ASOT that is accessible to all producers and consumers. Conceptual design data may be available in later design phases (e.g., preliminary or detailed design), however, in this case the data may be distributed.

In a distributed data architecture, the ASOT is decentralized so that multiple data sites exist. Distributed data architectures can be further subdivided into distributed data where identical copies are stored in multiple locations or data silos, where data is still stored in multiple locations without additional redundancy. To furnish data required by producers and consumers, a complex set of data transactions are required to transfer and synchronize data between the individual sites and producers/consumers that interact with them. Distributed data architectures are inherently more complex than centralized architectures, as data must be transferred between multiple sites, potentially requiring complex synchronization logic. While distributed data architectures may be too complex for connecting processes that have strong interdependence (e.g., within the same design phase), the Digital Thread may be viewed as a large distributed data architecture. In that case, disparate data locations may not only be feasible, but also desirable, as relevant data is stored where it is most frequently needed (e.g., within a design phase), while accepting the resulting complexity when accessing data outside of its context/phase.

It is also important to differentiate between a common schema, such as the one used by van Gent et al. [32], and a data architecture. A common schema does not infer any information about where the data contained within the schema is stored. As a result, it is conceivable that a common data schema may be used to parse data passed to a data consumer without any assumption on locality or associated complexity.

Also note, the difference a schema and an ontology: An ontology describes the concepts within a domain and how these concepts relate to each other. For example, an aircraft may contain a fuselage and a wing. The wing is attached to the fuselage. This may be represented as an ontology, capturing the relationships between the concepts, aircraft, fuselage, and wing. Each concept may have further properties that help describe it. A schema, by contrast, describes how data is stored. The data used to define a fuselage may, for example, be store in a SQL database with a fixed schema. This schema is a contract that expects certain values and allows for retrieval of this information. Ontologies are often most useful when combined with reasoning (e.g., does an aircraft contain at least one wing?).

Finally, note that the concept of data architectures and locality is a helpful construct to describe framework and application interactions. However, in practice, even centralized data architectures are likely highly distributed. For example, a centralized database may be hosted in the cloud using Kubernetes. In practice, the database is not strictly centralized, but uses sharding and distributed computing to ensure redundancy, prevent data losses, and availability for high-traffic scenarios. In this case, the cloud cluster abstracts the data distribution away from the user, reducing

complexity. Nonetheless, treating data architectures as a high-level description organization-level interaction can be useful for framework and study design.

B. A Holistic Approach to Data, MADO/MDO, and MBSE

Until recently, distributed MDO referred to architecturally distributed problems, while representing computationally distributed MDO was ill-defined or wholly undocumented. Lupp et al. [61] demonstrated that this lack of definition w.r.t. computational partitions can result in substantial slow-downs when incorrectly reproducing the work documented in a N^2 /XDSM diagram. As a result, they presented an extended nomenclature and concepts to accurately and wholly describe MDO problems. This section describes concepts established by Lupp et al. [61] and expands them to describe data and locality within a MDO problem.

Lupp et al. described a *discipline* as the most basic unit of a multi-disciplinary system. While discipline complexity may vary, each discipline fundamentally accepts inputs and produces outputs, mapping from $\mathbb{R}^N \rightarrow \mathbb{R}^M$, such that:

$$f(x_1, x_2, \dots) = [y_1, y_2, \dots] \quad (1)$$

A *group* is a multiplicity of disciplines. A *connection*, in the context of MDO, denotes a transfer of data from one discipline to another. A *partition* of a multi-disciplinary system is a computational grouping, containing one or multiple disciplines. The partition denotes the computational location of the disciplines. For example, a partition may describe a set of disciplines as located on a specific computer or server/cluster. Additionally, partitions can also be nested. For example, a nested partition may be an appropriate way to represent individual computers in a cluster.

Data may be exchanged within a partition or over its boundary. However, the transfer cost of the data may vary depending on the type of boundary. The coupling variables of the partition disciplines may be private and not exposed to the global multi-disciplinary system, allowing for the representation of data rights. *Data rights* are rules that govern which entities (and by extension which users or engineers) are permitted to view specific data.

Finally, Lupp et al. introduced the concept of the multi-disciplinary system *topology*. The *topology* of a multi-disciplinary workflow describes how a multi-disciplinary system is computationally structured (Figure 2a). This includes how coupling variables are grouped and to which disciplines they are visible. Furthermore, the *topology* may describe the physical location of the disciplines and discipline groups (i.e., for geographically distributed computing).

However, this past work does not account for data descriptors or data locations outside of disciplines. To extend these prior conventions, this work introduces the concept of a *data block*. A data block may be thought of as a discipline that does not provide inputs or a function evaluation; it merely provides output data (similar to *IndepVarComp* in OpenMDAO [62]), with the exception that it provides a data structure of arbitrary nesting (unlike OpenMDAO, which treats *IndepVarComps* as flat maps with one nesting level). It represents data storage and can be used to express a database or part of a database. Note, that a data block need not directly reflect a database's underlying structure (e.g., tables, schema, etc.), but may include a mapping layer that translates these to high-level schematic concepts.

While a data block introduces the concept of data structures, this alone does not describe data locality, architecture, or data rights. However, data locality can be expressed by including a *data block* within a computational partition (Figure 2b). Now the data is clearly represented not only within the N^2 , but also exactly in which computational boundary it resides. Likewise, data rights may be represented the same way they are represented between individual disciplines. For example, the connection between *Data Block 1* and *Discipline 2* restricts data rights, so that other users and services may not be able view which data (or if any) is exchanged. Likewise, *Data Block 2* and *Discipline 4* are connected with restricted data rights. In practice, *Data Block 2* may not be visible to the remaining topology (e.g., an internal database).

As a result, these new definitions provide unique and reproducible information on how and where data is stored within an MDO problem. The introduction of these data concepts now directly provides a unification of MDO and data, data locality, and data rights required to connect to the ASOT and MBSE.

C. Mapping Data between MBSE and MADO/MDO

Accurately and reliably describing data locality, data flow, and data rights within an MDO problem is a necessary condition to integration between MDO, the ASOT, and MBSE. However, while MDO and MBSE ultimately process the same data, they represent it differently. The data representations previously introduced into MDO provide a strict structure of inputs and outputs (from disciplines), which may not exist in this format in an MBSE template. This is especially true when using a GRA-like format as the basis of a MBSE model. Varying MDO studies may result in an input being reclassified as a driven variable or output in another. For example, wing area may be a design input for certain analyses, while being a derived quantity/output for a differing wing parameterization.

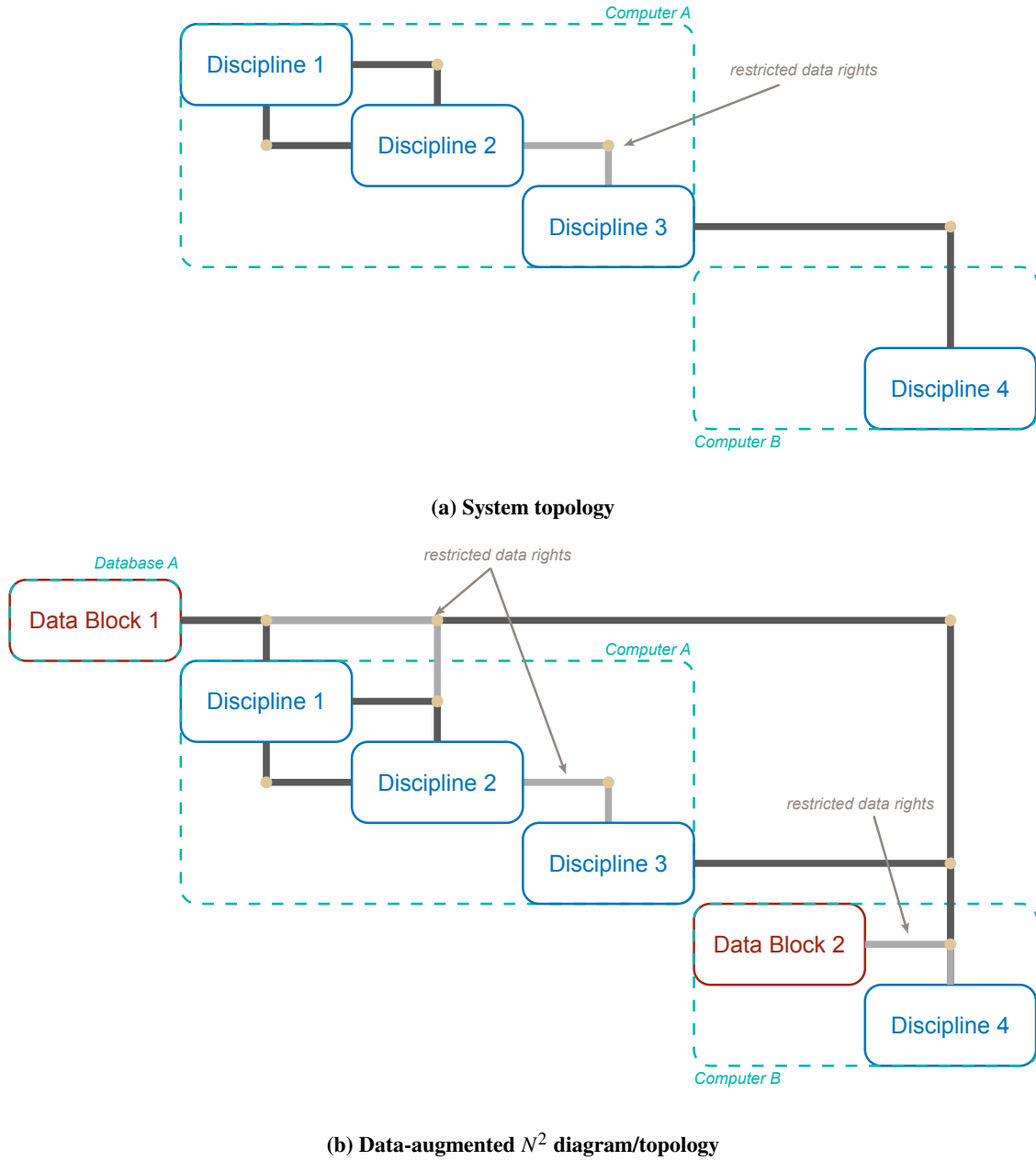


Fig. 2 Illustration of a system topology containing the system's partitions, disciplines, and connections (top). The data-augmented N^2 diagram (bottom) expands the graphical representation of a topology to include data stores and structures.

The full MBSE model may include both inputs and quantities calculated within the multi-disciplinary system. Within the MDO representation, these quantities are distinct. As a result, MBSE-MDO integration requires a mapping of the architecture model (MBSE) to the MDO representation, which may be expressed mathematically as:

$$\mathcal{M} : (K_{MBSE}, v \in \mathbb{R}^N) \rightarrow (K_{MDO}, v \in \mathbb{R}^N), \quad (2)$$

where K_{MBSE} and K_{MDO} are the MBSE and MDO keys/variable path names, respectively, and v is the variable value. Note, that the value v is unaffected by the map. Naturally, the map $\mathcal{M}$ is invertible, so that the mapping MDO to MBSE is possible. Furthermore, this mapping must also include metadata to help differentiate between inputs and outputs within the MBSE model. This mapping allows MBSE and MDO representations to exist separate of one another, serving as a translation layer between them.

III. Creating a Generalized Authoritative Source of Truth

This work creates a central authoritative source of truth for diverse engineering workflows, Nucleus. This section details the types of data needed to capture engineering analyses and how they drove the database architecture. Ontological concepts implemented in the database are presented, as are the relationship of the data repository to the larger Digital Thread.

A. Architecture and Implementation

Fundamentally, engineering analyses contain two different types of data: structured and unstructured. Structured data follows a consistent format (e.g., lift coefficients, fuel burn, etc.) and can therefore be parsed and searched easily. PostgreSQL, MongoDB, MySQL, among many others, are databases that can store structured data (albeit using a variety of paradigms).

Unstructured data, by contrast, does not follow a consistent format (e.g., files), making them harder to search. Nonetheless, engineering data usually consists of a combination of structured and unstructured data. For example, analyses will provide structured data such as lift coefficients, weights, etc., but also output files that may be needed for post-processing and interpretation.

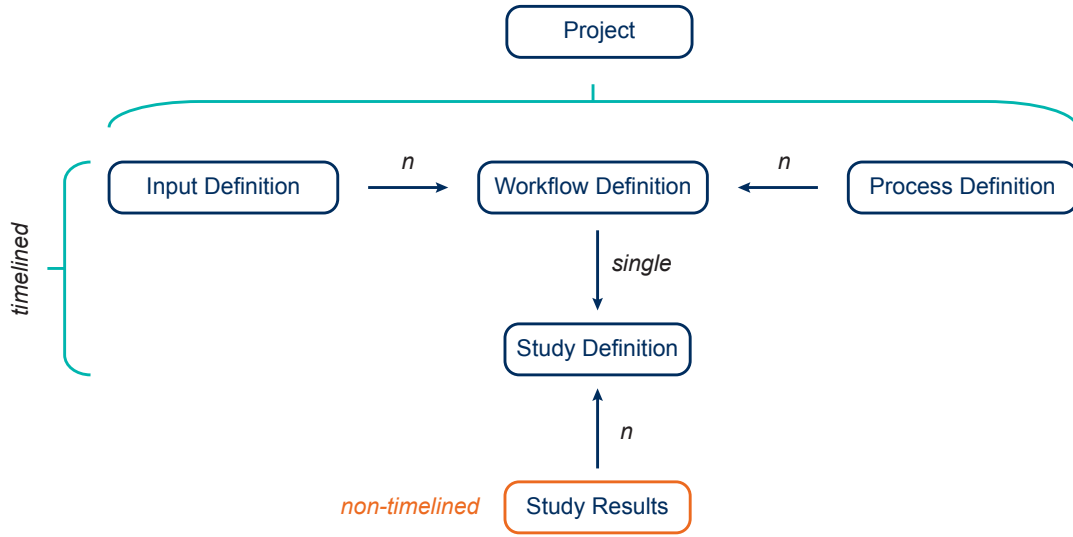
As a result, Nucleus uses a bi-modal architecture to represent both types of data: MongoDB for structured data and MinIO (a S3 compatible object store) for unstructured data. MinIO (and S3 compatible storage in general) are commonly used to store files in web and cloud applications. Structured databases are largely classified into SQL and NoSQL databases. SQL databases use predefined schemata, while NoSQL databases offer more flexibility by permitting a dynamic schema. MongoDB was selected for Nucleus due to a need for dynamics schema (see Section III.B). While both types of data (structured, unstructured) can be stored within Nucleus, unstructured data is “registered” in MongoDB, including metadata, to make it more easily searchable and linkable to structured data. An object relational model, Pydantic, is used to map database concepts between Python and MongoDB, while also enforcing data validation.

Nucleus itself provides a REST application programming interface (API) (network) layer on top of these databases. Many engineers do not have experience with databases or their respective query languages. As such, the API was designed for use by engineers without deeper training in database concepts. The entire application is designed using the microservice paradigm, with all components of Nucleus using containerization. Deployment is achieved using Kubernetes, a standard deployment tool for cloud and microservice applications. The use of cloud technologies enables Nucleus to be deployed to a large variety of infrastructure (from on-premises to cloud), while being able to scale horizontally and elastically to meet large and small traffic demands.

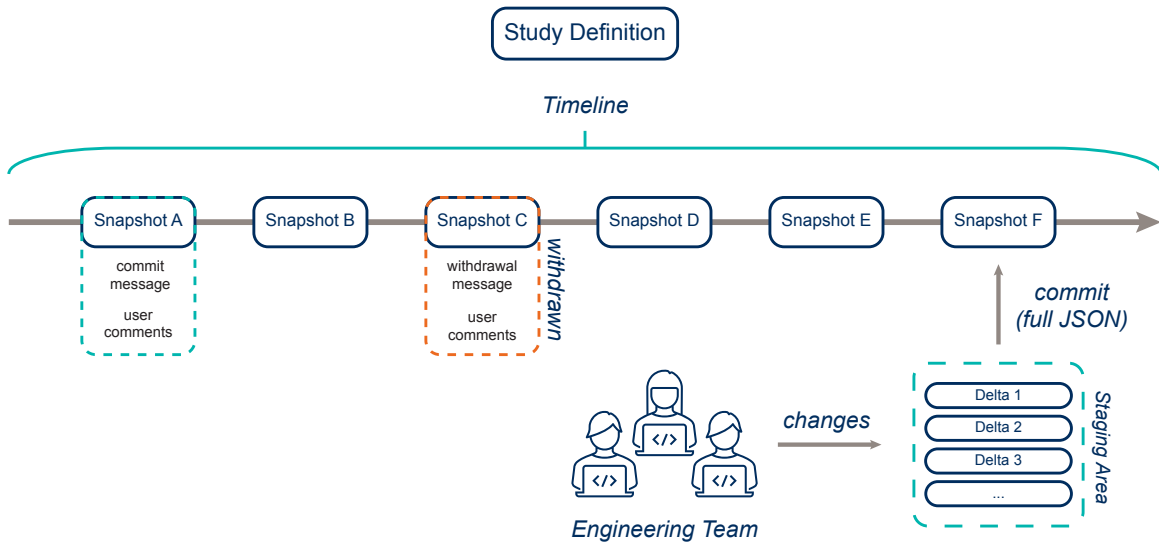
B. Ontological Concepts

While structured data has many practical advantages over unstructured data, the structure is imposed by ontologies and schemata. Ontology and schema are often used interchangeably, though they are not necessarily the same. A schema defines the structure of data (e.g., how a database table is structured), while an ontology more broadly defines a domain of knowledge and relationships therein. While Nucleus utilizes schemata to organize data—schemata within the database are dynamic due to the nature of the information stored—discussing the ontological concepts that form its foundation is scientifically more useful.

Fundamentally, there are two distinct sets of ontologies that exist within Nucleus: project-level ontologies and a top-level ontology that governs all data within the data repository (Figure 3a). Nucleus was designed for generality, so the project-level ontology is problem-specific (e.g., in REACT driven by the MIL-STD-881 work breakdown



(a) Relationship of ontological concepts



(b) Resource timeline

Fig. 3 Representation of ontological relationships and timeline implementations for resources within Nucleus, based on the example of a Study Definition

structure (WBS)). This also drove the need for a dynamic schema in the structured database, resulting in the selection of MongoDB (NoSQL). Various GRA that describe aircraft and their subsystems are examples of project-level ontologies.

By contrast, the top-level ontology for Nucleus (Figure 3a) defines entities and how they relate to each other in a more general form. At the global level, Nucleus introduces the following entities: *Projects*, *Input Definitions*, *Process Definitions*, *Workflow Definitions*, *Study Definitions*, and *Study Results*. Together, these concepts describe an end-to-end engineering workflow that includes inputs, how analyses are evaluated, the scoping of the study itself, and study results.

A *Project* is a top-level concept that contains all other concepts. A *Project* may contain a multiplicity of *Input Definitions*, *Process Definitions*, *Workflow Definitions*, *Study Definitions*, and *Study Results*. *Projects* were purposefully scoped to encompass multiple studies and aim to reduce unnecessary reproduction by engineering users.

Input Definitions capture inputs to the engineering problem. They are generally data structures of arbitrary nesting depth. Like other Nucleus resources, they are designed for composability and reuse, reducing repeated workloads for the engineering team. As a result, teams may break down inputs into multiple *Input Definitions* and reference them as needed into *Workflow Definitions*.

The definition of an analysis or MDO workflow is created using *Process Definitions* and *Workflows*. These concepts are related, although they are differentiated by important nuances. A *Process Definition* declares which disciplines are grouped together. Like groups in the MDO-sense, *Process Definitions* can be nested. *Workflows*, by contrast cannot be nested and are designed to be the final workflow executed by a study. They can contain multiple, nested *Input Definitions* and *Process Definitions* (Figure 3a).

A *Study Definition* declares the scope of a study. A single *Workflow Definition* is referenced and can be used with a variety of drivers (e.g., optimizers, DOE, etc.). Objectives, design variables, and constraints can be defined within the *Study Definition*. These definitions allow for a modular representation of studies: e.g., a single *Workflow* can be referenced and used by multiple *Study Definitions*, as in practice studies may keep the general input data structure unchanged while only modifying design variables, constraints, etc.

Finally, *Study Results* contain structured and unstructured data from the run study. They directly reference a specific version of a *Study Definition*, ensuring that data is directly linked to the definition that created it.

C. Relationship to the Digital Thread

The concepts introduced in Section III.B are sufficient to describe and document studies for a wide variety of engineering workflows. However, they remain insufficient for integration into a Digital Thread. A primary goal of REACT and Nucleus was to ensure traceability of designs and design decisions. Several of the project-level ontological concepts therefore are timelined (Figure 3a).

Each such resource contains a timeline that creates a version-controlled record of its changes (Figure 3b). Each timeline has a staging area, similar to git, that is shared between all users on an engineering team. Changes can be added to the staging area. The API enforces conflict resolution by running a validation check for every submitted change. While the staging area tracks individual changes and atomic changes can be added and removed as necessary, once the full set of changes are committed to the timeline, they are stored in a monolithic JSON representation. Like git, each commit requires a commit message to create a snapshot. To further enhance traceability, other users can add comments to snapshots over time. In case resources reference each other (e.g., to define a study), the reference points to a snapshot so that the relevant configuration for a study is clearly identifiable and the study can be uniquely traced from inputs to results.

While snapshots may not be deleted, they can be withdrawn to indicate users should not use them. Withdrawals also require messages indicating why a snapshot was removed from circulation. While Figure 3b shows a timeline for a *Study Definition*, the rest of the timelined resources in Figure 3a follow the same timeline logic.

The traceability that Nucleus offers is key to understand how designs change over time and for what reasons. It offers a solution tailored for early design phases, which have been difficult to capture within the Digital Thread. Rather than attempting to represent the entire Digital Thread, Nucleus specifically addresses research and development, conceptual, and preliminary design phases (Figure 4). Commercial tools exist for tracking detailed design, manufacturing, and sustainment. By contrast, these tools are inappropriate for earlier R&D and design stages, leaving a gap in the Digital Thread. The current gap may result in early design decisions being lost to later stages of the program. In this context, Nucleus' role is to add traceability to early design stages and allow decisions to feed forward, potentially to other organizations.

As mentioned in Section II.A, data locality may depend on the level of abstraction of the observer. Nucleus, clearly implements a strongly centralized approach to data. All data producers and consumers send and obtain data directly

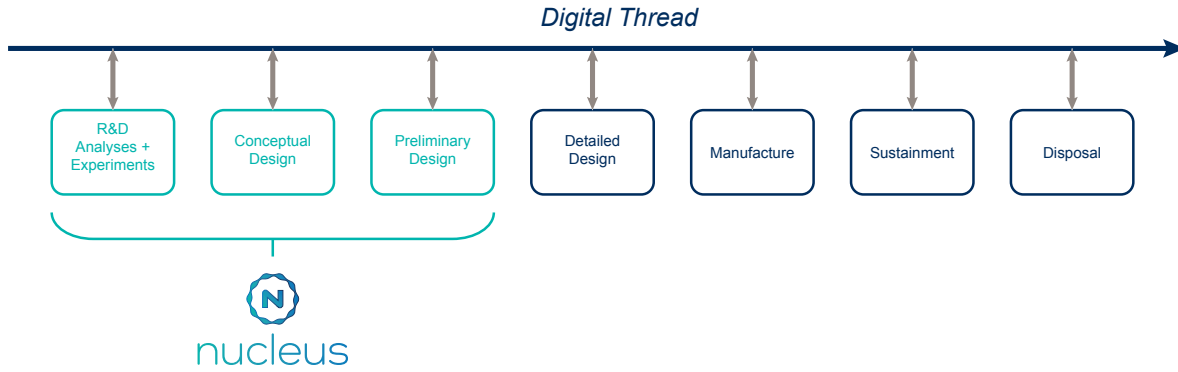


Fig. 4 Conceptual representation of the Digital Thread

to/from Nucleus. However, in the context of the Digital Thread, Nucleus is only one of many data locations. While this does increase complexity of maintaining and referencing data, this architecture is inevitable for realistic problems at large organizations. Maintaining a centralized data architecture for individual design phases is reasonable, thereby reducing the complexity for data transfer and synchronization for frequent operations. The increased complexity of other data consumers outside of the design phase querying Nucleus data is a necessity, but remains manageable due to its relatively small volume. For both cases, Nucleus offers a standardized, open API that allows any data consumer to retrieve information as needed.

D. Authentication and Data Rights

Nucleus is intended to be an enterprise-grade datastore that serves as the ASOT in MBSE-MDO workflows. As such, strict requirements regarding user/service authentication and data rights enforcement must be fulfilled. The Department of Defense (DoD) provides guidelines and requirements that software must pass to receive an Authority to Operate (ATO).

Nucleus utilizes OAuth2 and OpenID Connect (OIDC), standard authentication protocols that fulfill ATO requirements. OIDC provides user authentication via Common Access Card (CAC) authentication, while OAuth2 enables trusted services (such as MDO frameworks, etc.) authorization to read or write data to the database.

At the time of writing, Nucleus does not feature active role-based access control to enforce data rights. However, it was designed from inception to support data rights at an atomic level. Therefore, with the visibility of individual pieces of information/variables within *Input Definitions*, MDO groups and partitions can be controlled depending on the user/service accessing the resource. Only authorized users will be allowed to view information. Furthermore, the workflow/ N^2 definitions within Nucleus feature the same atomicity and, being composable and separable, allow only authorized parts of the workflow to be presented to a user. Parts of the workflow that the user has no rights to view are simplified to abstract the workflow without making it unusable.

IV. Numerical Studies

The numerical studies in this work provide four vignettes to illustrate Nucleus' capabilities and role in establishing a Digital Thread for R&D, conceptual design, and early preliminary design. The first vignette presents a DOE study during early conceptual design, using CREATE-AV ADAPT [36, 37], for a notional blended wing body (BWB) aircraft. The second study demonstrates a gradient-based multidisciplinary optimization of the same BWB aircraft. While the fidelity of the analyses used in the optimization are low (vortex lattice method (VLM)), the pairing of the early conceptual design and the MDO example is also intended to illustrate transitioning from one design cycle/phase to another, while maintaining traceability on the Digital Thread through Nucleus (and potentially MBSE). As in a realistic transition, the early conceptual study and optimizations were run by different engineers, without prior knowledge of the tool the other used.

Meanwhile, the third and fourth vignettes illustrate collaborative engineering scenarios. The third example centers on two engineers working together to perform high-fidelity aero-structural analyses. The high-fidelity results are

synchronized with Nucleus to illustrate its ability to serve as the ASOT for both large scale data and data of varying degrees of fidelity. Finally, the last vignette demonstrates the ability to interface between MBSE and MDO/design via Nucleus. Two geographically separated engineers, a stakeholder and an aircraft designer, modify a notional aircraft to fulfill requirements defined by the stakeholder. This example illustrates both distributed teams' ability to effectively interact using the ASOT as well as the input and evaluation of requirements by stakeholders within the REACT framework while interacting with design engineers.

A. Early Conceptual Design of a Blended Wing Body Aircraft

To enable the first study, a CREATE-AV ADAPT [36, 37] plugin was created that enabled ADAPT to pass and receive data from Nucleus (the initial version of the plugin was created in a collaboration with NAVAIR and CREATE-AV by AJ Field and his team). The plugin allows ADAPT users to add data synchronization blocks to ADAPT workflows, thereby enabling direct exchange of data between the tools.

The first vignette explores Nucleus' utility in early conceptual design and concept exploration of a notional BWB (Figure 5a). It also illustrates the integration of a standalone executable or analysis. Nucleus does not require every tool that interacts with the database to be a part of a larger, integrated framework. The workflow (Figure 5b) involves pulling data from Nucleus, running an ADAPT DOE, and then synchronizing the resulting data back to the database.

To achieve this technically, a pair of new ADAPT generic plugin blocks were developed. The first of these was the database receive block. In it, the block pulls down a user-supplied JSON dictionary from the project object in Nucleus that maps process input variables to their respective ADAPT variable counterparts. In addition, the plugin also pulls down the most recent version of the process input variables from Nucleus via the REST API. Once obtained, the plugin then goes variable by variable updating the values in the ADAPT model to reflect those in the database. This can be run in either the pre-processing or workflow execution stage of the ADAPT run. As would be expected, the other block required was a database send block. In an inversion of the previous block, the send block pulls down a user-supplied JSON dictionary from the project object in Nucleus that maps specified ADAPT variables to specified process output variables. Once again, the block works through the map variable by variable, saving the ADAPT model values for each variable into a Python dictionary. Once the end of the map is reached, the aforementioned dictionary object is passed to Nucleus as a study result object via the REST API. Unlike the receive block, the send block can only be run in either the workflow execution or post-processing stage of the ADAPT run.

In order to exercise the capabilities of the ADAPT Nucleus plugin, a notional conceptual design workflow was created. For this workflow, empirical methods contained in the mission analysis software FLOPS were used for weight and drag estimation. Built-in analytical estimation methods within ADAPT were used for lift estimation and stored input was used for propulsion performance and C_{Lmax} estimation. These variables were passed into the mission analysis section of FLOPS, which sized the total fuel weight of the vehicle for a specified payload weight a specified distance.

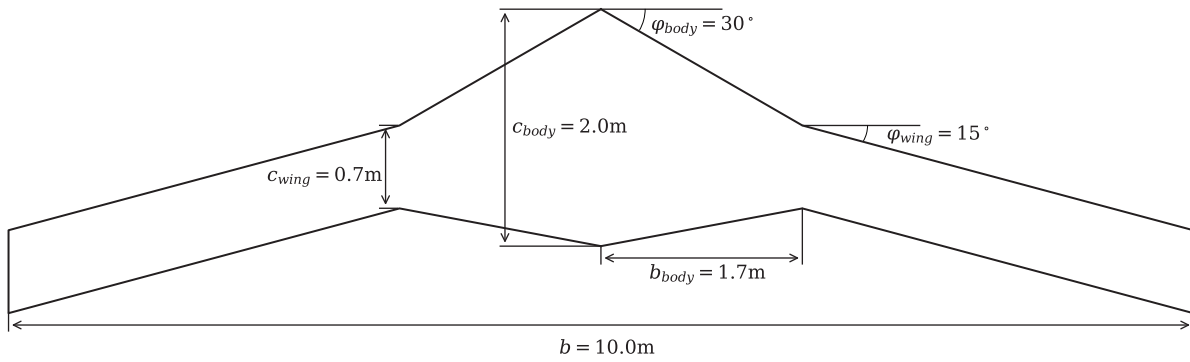
Once the workflow was set, the design sweep capabilities of the ADAPT software were utilized in conjunction with the the database blocks to complete a design sweep around a notional point. To do this, an initial project, input definition, workflow definition, and study definition were defined in Nucleus, with an array of five wing spans set as the input design variables. From there, an ADAPT sweep run was started locally, with the database automatically updating the span values of the outboard wing panel and uploading the resulting vehicle weight and cruise CD0 for each study. The results of this sweep are included in Figures 5c-5d.

This example demonstrated the utility of the data-augmented N^2 to describe the interaction of data and MDO processes. Even in this simple example, it clearly describes what data is being stored or provided, with data locations explicitly included in the N^2 . Furthermore, it also clearly articulates what data must be moved across computational boundaries. This is particularly important for problems that involve large data sets.

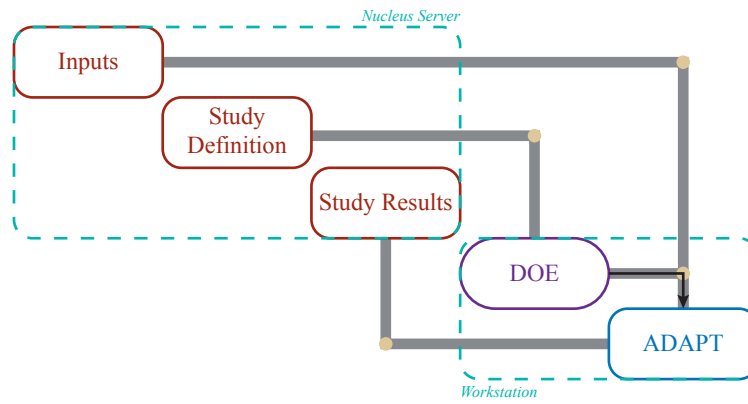
B. Multidisciplinary Design Optimization of a Blended Wing Body Aircraft

The second vignette describes a gradient-based optimization of the same BWB described in the first vignette. This second design study can be viewed as a surrogate for a new design phase that starts after the early conceptual study shown in the first vignette. As such, designers in the new design phase are able to easily review design decisions made previously by querying the records in Nucleus.

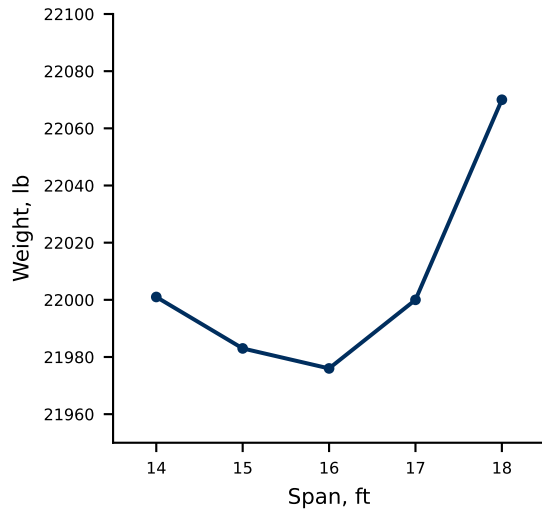
Differing from the previous study, this vignette includes a rudimentary aeroelastic model and structural sizing, thereby increasing analysis fidelity. A VLM solver, OpenAerostruct [63], serves as the aerodynamic solver, while a linear beam approximates the static structural response (Figure 6). OpenAeroStruct was created at the University of Michigan and uses OpenMDAO to set up and solve the VLM governing equations. It offers mesh manipulation utilities,



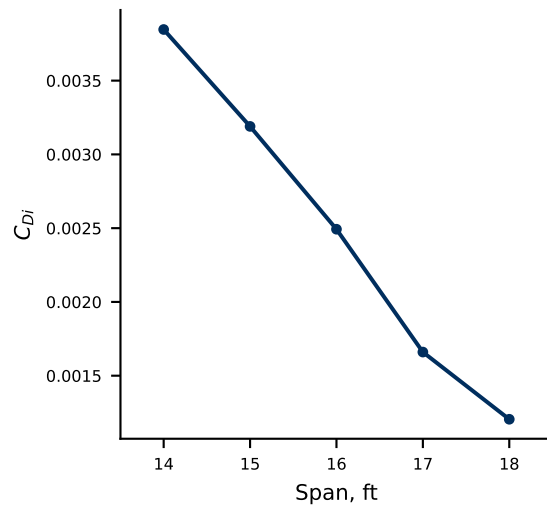
(a) BWB planform and planform variables



(b) Data-augmented N^2 for the early conceptual design studies of the BWB



(c) BWB weight



(d) BWB induced drag

Fig. 5 ADAPT workflow and sweep results for the early concept evaluation of the BWB

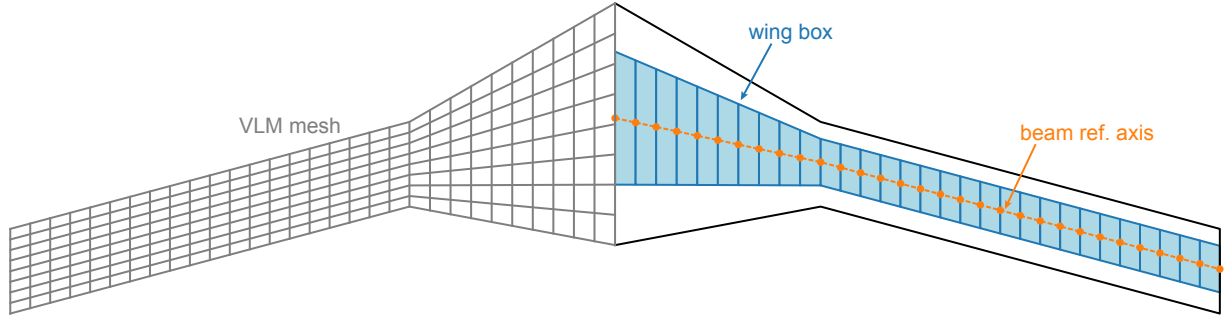


Fig. 6 Planform and analysis geometry of the baseline BWB model.

however, these are not used within this work, instead employing custom components to re-mesh the planform for every design iteration (Figure 7). While OpenAeroStruct is useful for simple studies, it is not representative of state-of-the-art industry tools. Within this work it serves as a surrogate and could be replaced with higher-fidelity analysis methods.

While the BWB model can support detailed weight analyses based on structural and component sizing, Raymer’s weight estimations for fighter aircraft [64] are used within this work (Figure 7). Raymer’s equations were determined from statistical regressions of existing designs. It reasonably predicts the weight of conventional configurations, but may not accurately predict the weight of the BWB. Despite the possible errors associated with using this method for this work, it provides a simple model to account for weight and enable design trade-offs.

In this work, a parametric engine model proposed by Torenbeek [65] is used to approximate the engine thrust, weight, and fuel consumption. A loiter optimization is conducted for a notional mission. The full optimization formulation is given by:

$$\begin{aligned}
 &\text{minimize:} && m_f, \\
 &\text{with respect to:} && \\
 & && x = [b_{wing}, c_{wing}, b_{body}, c_{body}, \alpha_b, t_i, T_0, \mu, \Pi, \zeta_t]^T, \\
 &\text{subject to:} && \\
 & && L_{VLM} = W, \\
 & && V_{fuel} \leq V_{wb}, \\
 & && KS(\sigma_{trim}) - \sigma_{yield} \leq 0.
 \end{aligned}$$

Inputs and options for the optimization were set up in Nucleus by creating *Input Definitions*, *Process Definitions*, *Workflow Definitions*, and a *Study Definition* in preparation for the optimization. The optimization was run using OpenMDAO [62] on an engineering workstation. A custom OpenMDAO data recorder object was created and added to the optimization problem that enabled OpenMDAO to write results to Nucleus as the optimization progressed.

Engineers were able to observe result data while the optimization ran, as well as the full data set after it completed. The results of the optimization are shown in Figure 8 and 9 and were post-processed from structured data saved to Nucleus during the optimization. The optimizer increases the size of the vehicle to increase fuel volume, while simultaneously increasing the engine bypass ratio and decreasing the engine’s static thrust. The reduction in static thrust likely occurs due to a direct correlation between static thrust and engine weight (due to a simplistic weight model) and due to the lack of a take-off or climb constraint that would provide a forcing function towards higher thrust values.

Despite its simplicity, this study illustrated Nucleus’ ability to support MDO/optimization workflows. In the case of a design team, engineers would be able to inspect the data as the optimization runs, start to post-process and interpret it, and make decisions based on the data they observe.

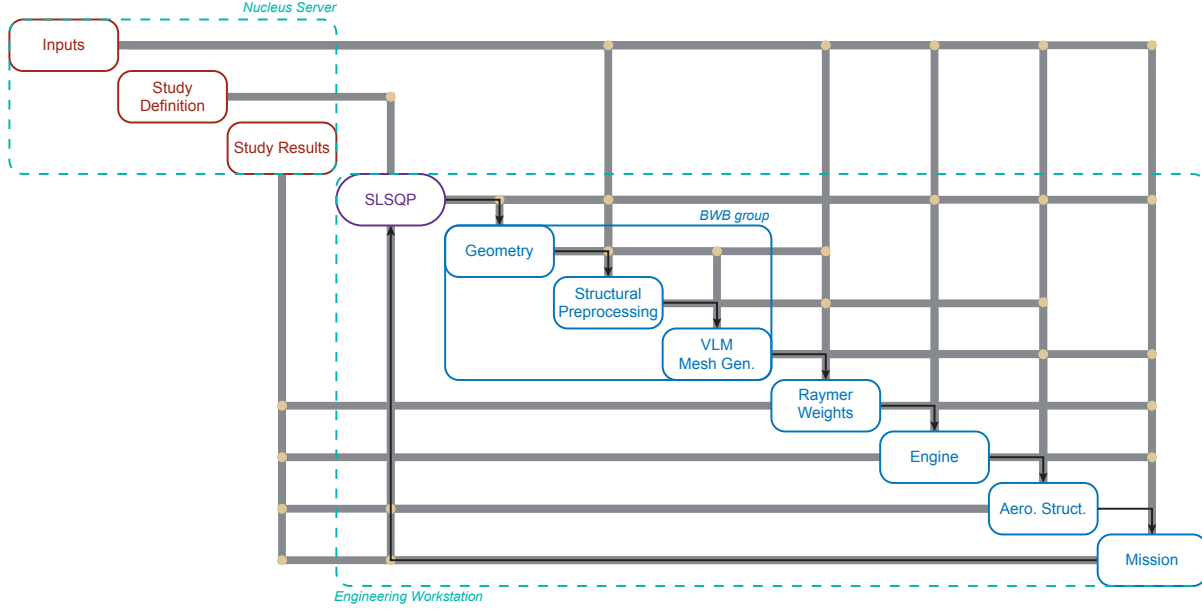


Fig. 7 Data-augmented N^2 for the BWB optimization problem.

C. High-Fidelity Fluid-Structure Interaction

The third vignette describes the interaction of two engineers setting up and evaluating high-fidelity aeroelastic simulations. Leading up to this study, teams within MSTC worked to integrate OptiMSTC [66], AFRL/RQV's in-house high-fidelity aeroelastic analysis and optimization framework, with Nucleus as a central source of truth.

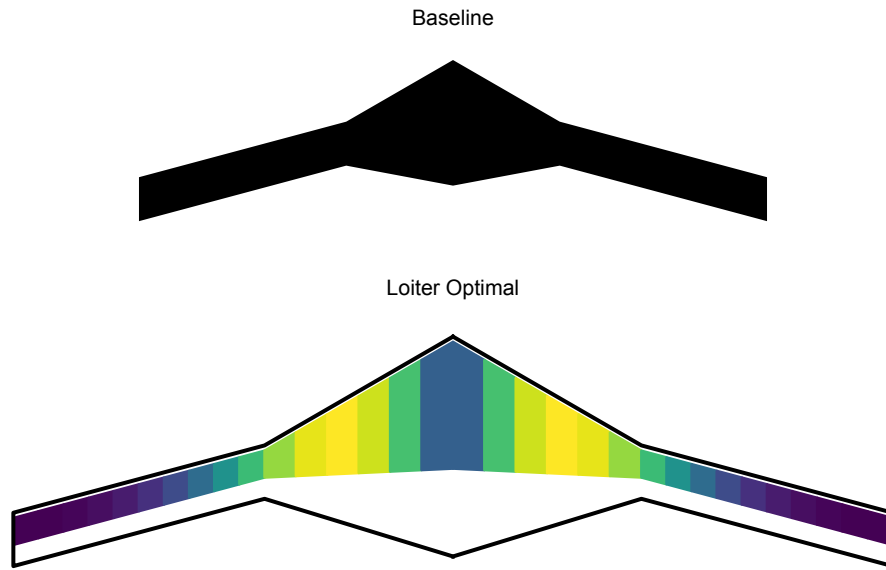
OptiMSTC is a Python based tool that aims to simplify the problem setup and coupling between physics disciplines. It relies on ESP/CAPS for geometry modeling, mesh generation, and analysis input file writing. Toolkit for Analysis of Composite Structures (TACS) is used as the structural finite element solver while NASA's FUN3D serves as the CFD solver. Load and displacement transfers between the two disciplines are handled by FUNtoFEM. A typical analysis using OptiMSTC requires a user to provide a geometry file and to specify any design variables, analysis inputs, optimization settings, and objective and constraint functions via Python dictionary. OptiMSTC will then process these dictionaries, build the geometry, create analysis input files, and then set up and run the analysis or optimization driver automatically.

To integrate Nucleus into the existing OptiMSTC workflow only three additional functions need to be added. The first function utilizes the same set of Python dictionaries that define the OptiMSTC problem to populate a project in the REACT database. First, design and configuration variable values, units, and data types are posted to a new database *Input Definition*. Next a *Process Definition* is created within Nucleus, with all design and configuration variables added as inputs and objective and constraint functions added as outputs. A workflow is defined within Nucleus to link the previously created input and process definitions. Finally, a new study is defined within Nucleus that includes the type of study to be run, bounds for each variable function, and any optimization driver options. After each of these resources have been created and committed to the database timeline, the ESP geometry files are uploaded as project artifacts. The populated database now contains all the necessary information needed to reconstruct and run an OptiMSTC problem.

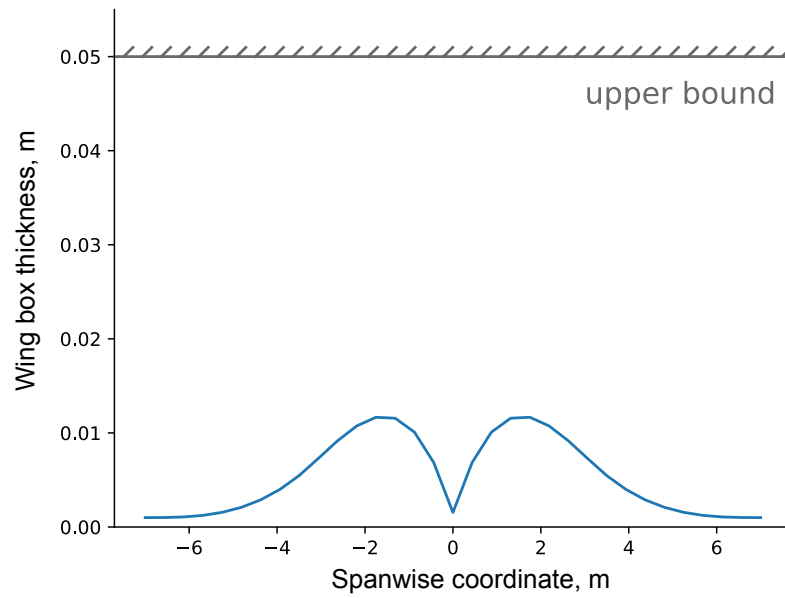
The second newly created function reverses the process of the first function by pulling down the project data and reconstructing the OptiMSTC Python dictionaries that created an analysis. After Nucleus' authentication, the geometry artifact files and study definition are retrieved from the database and used to initialize and then run the OptiMSTC problem. Once the analysis or optimization has been completed, the results are processed and uploaded as a new study result within Nucleus. Any output files generated during the analysis (e.g. Tecplot files from FUN3D) are also uploaded as study artifacts (stored Nucleus' object store).

The final utility function implemented in OptiMSTC retrieves study results that are present in Nucleus. Given user authentication, the project name, and the study name, this function automatically downloads each study artifact and the displays the study results as a table.

The integration task was simplified by Nucleus' open API implemented in REST that abstracts data to concepts

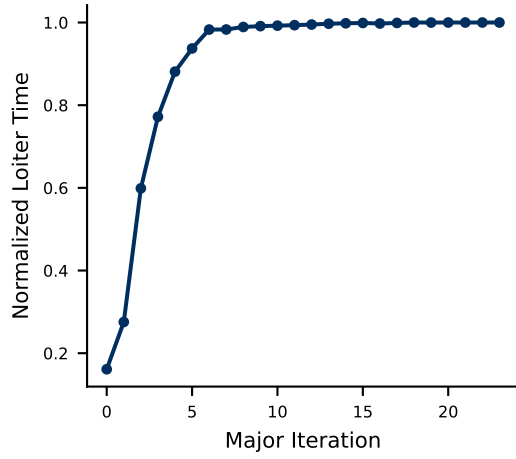


(a) Wing planform with optimal wing box

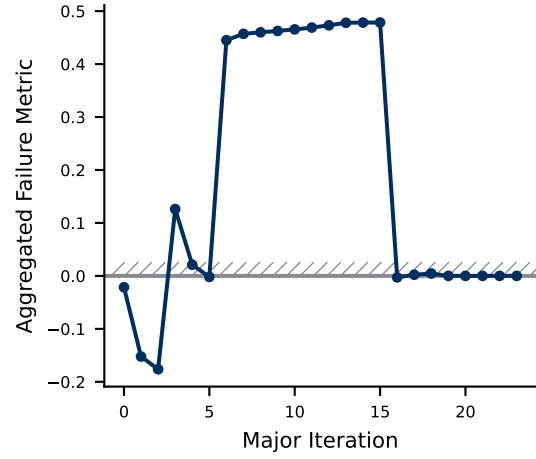


(b) Thickness distribution

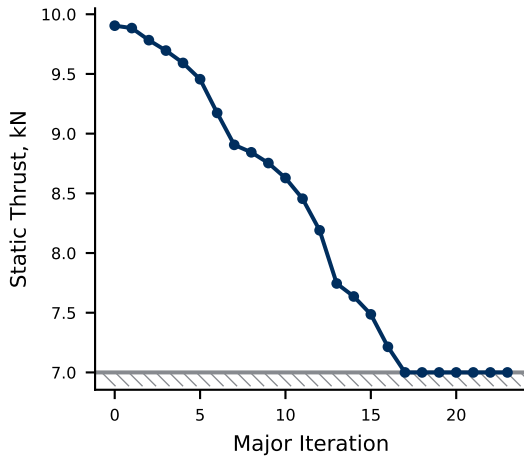
Fig. 8 Optimal BWB wing box thickness distribution



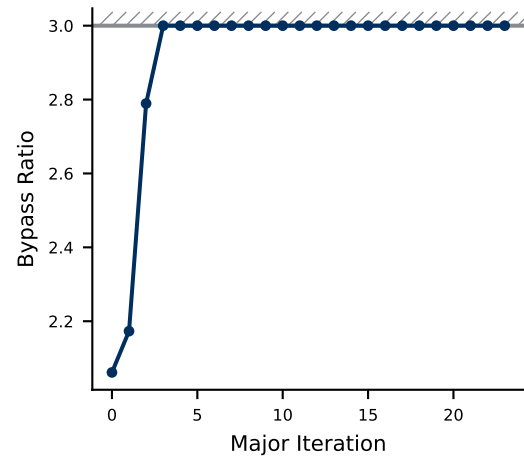
(a) Loiter time (objective)



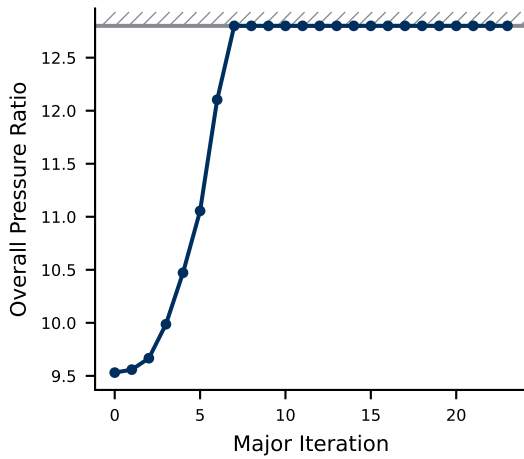
(b) Failure metric (constraint)



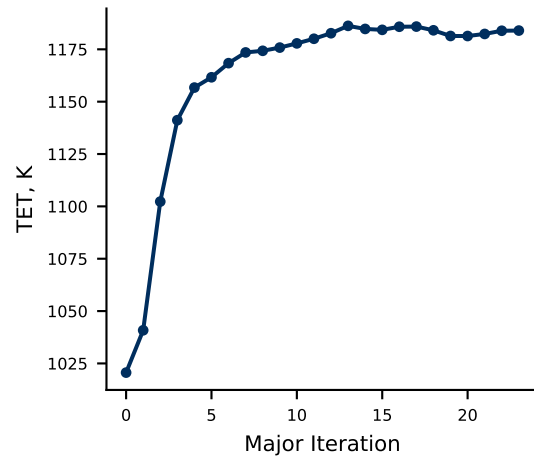
(c) Static thrust



(d) Bypass ratio



(e) Over-All Pressure Ratio (OAPR)



(f) Turbine Entry Temperature (TET)

Fig. 9 Iteration history for the objective, failure constraint and engine design variables of the BWB gradient-based optimization.

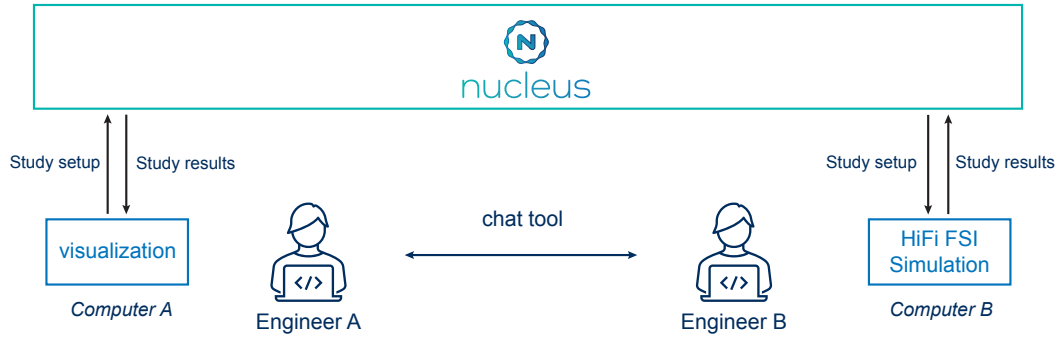


Fig. 10 Architectural diagram of the high-fidelity aeroelastic study.

engineers are familiar with. Furthermore, the team members conducting the integration had no prior experience with REST or RESTful APIs, Nucleus’ ontologies, or Nucleus itself. Despite this, the task was completed within a couple weeks part-time work, illustrating the low bar of entry Nucleus and its API present.

This vignette features two engineers, *Engineer A* and *Engineer B*, tasked with conducting a high-fidelity aeroelastic study of a notional vehicle (Figure 10). Both engineers are involved in the interpretation of the study results and require access to study definitions (parameters, options, geometry, etc.) as well as results. They rely on the Nucleus database as the central authoritative source of truth to synchronize data.

The numerical results were presented by Sandler and Novotny [67], investigating the Aeroelastic Optimization Benchmark (AOB) [68]. The optimization statement for the AOB case one is to minimize the structural mass of a wing based on the Boeing 717. 2.5g pull-up and 1g push-down maneuvers are analyzed, with each subject to an aggregated failure constraint and a lift trim constraint. The wing structure includes two spars, twenty-three ribs, and upper and lower skin panels between each rib pair. The spar, rib, and skin panels are constructed from composites and contain stiffeners. Each panel has a unique design variable for base thickness, stiffener height, and stiffener thickness. These design variables are additionally constrained to limit the change in thickness and height between adjacent panels. Altogether, this represents an optimization problem statement with 665 design variables and 1030 constraints.

Engineer A created the Python script defining the problem and the ESP geometry model. Once this was completed, *Engineer A* used the new Nucleus integration utility to create a matching project in the database and upload geometry artifacts (Figure 10). Next, the project and study names were communicated to *Engineer B*, who created a version of the OptiMSTC problem on their local computer by pulling from the database. The aeroelastic analysis was run on *Engineer B*’s machine and the results were synchronized with Nucleus. *Engineer A* was then notified via a chat program that the results were ready for viewing and subsequently downloaded the study results and artifacts to their computer. *Engineer A* was then able to open the structural analysis Tecplot file, visualize the results, and perform any post-processing on the data without needing to run the analysis or install any of the analysis tools.

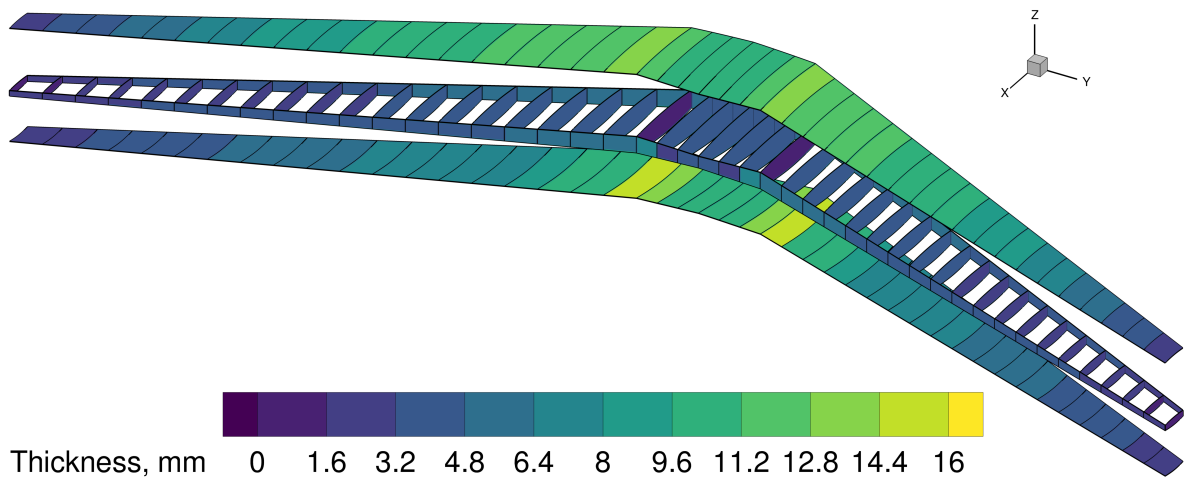
This study illustrated the use of a central data repository (Nucleus) to enable a collaborative workflow between a group of engineers. Furthermore, it demonstrated the ability working on a high-fidelity study.

It should be noted, that the same tenets of this vignette could be applied to a larger MDO study. In that case, teams representing disciplines (or groups of disciplines) could independently push their parameters to the central database before the larger, coupled study was executed. As with this study, real time communication, ideally via an instant messaging platform, is essential to coordinate between the team.

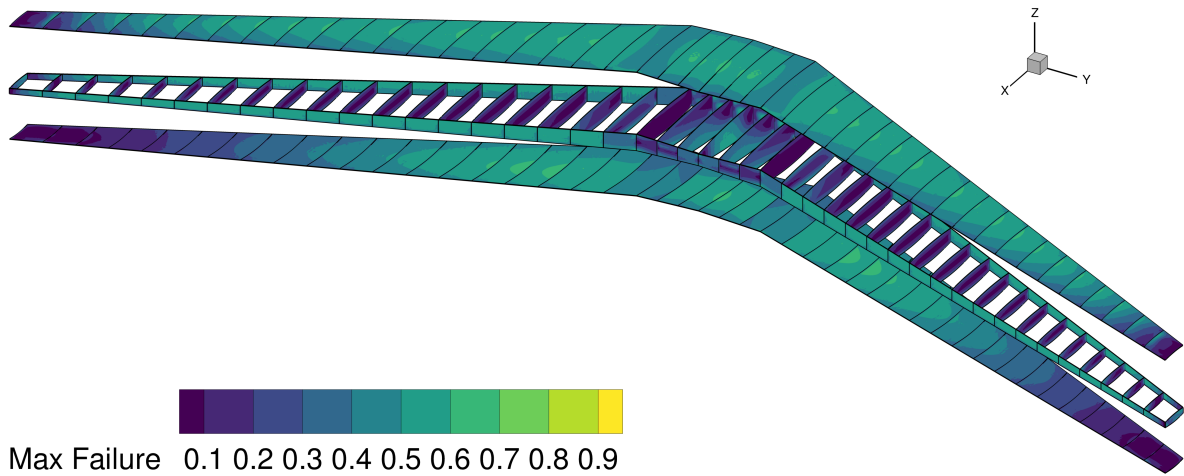
D. Demonstrating MBSE/Design Integration

The final vignette was chosen to demonstrate Nucleus ability to support MBSE-MDO coupled workflows. As with the previous example, this study involves multiple engineers that are tasked to solve a problem collaboratively. In this case, a system engineer provides requirements, while a designer evaluates the aircraft performance. The description of this vignette is based on a 2022 demonstration by the REACT project conducted by Marco Kobayashi (University of Dayton Research Institute/AFRL on-site contractor) and AJ Field (NAVAIR and CREATE-AV). As the REACT framework has evolved and improved, this demonstration has been repeated multiple times with an expanded scope.

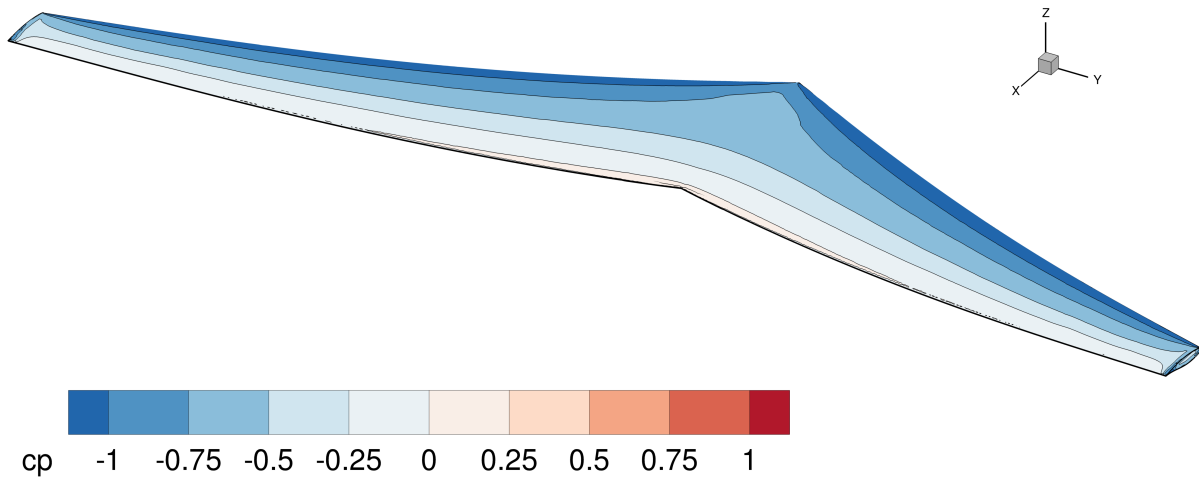
This example required the development of a MBSE/Cameo plugin that interfaced with Nucleus to synchronize data.



(a) Thickness distribution of the wing box.



(b) Distribution of the von Mises failure criterion.



(c) Aerodynamic pressure coefficients for the fluid-structure interaction test case.

A survey of available languages that can interface with Cameo was conducted and Groovy was chosen to create a proof concept. Jython was considered but rejected due to its foundation on the unsupported Python 2.7. Once a proof of concept was demonstrated, the REACT team started developing a true Cameo plugin using Java. The Java API allows developers much tighter integration into Cameo and offers features that the other Cameo interface languages do not.

As in the other studies, Nucleus serves as the ASOT for this study. It also serves as a bridge between the MBSE environment (Cameo) and the aircraft conceptual design tool (CREATE-AV ADAPT). Each tool can push and pull data to/from Nucleus (Figure 12a). Furthermore, a mapping exists that converts data structures from MBSE to MDO conventions and vice versa.

Two engineers, *Engineer A* and *Engineer B*, are collaborating to evaluate whether an aircraft fulfills stakeholder requirements, and if not, attempt to modify the design. *Engineer A* is a systems engineer and interfaces with Cameo, while *Engineer B* is a conceptual designer working with CREATE-AV ADAPT. Adding to the complexity, the engineers are geographically separated (Figure 12b, relying on Nucleus as the central source of data exchange. Both have access to a business instant messaging system, allowing them to coordinate or alert the other regarding actions they took.

The demonstration begins with the systems engineer inspecting the MBSE model. They have been given requirements on the maximum take-off distance of the aircraft and need to ensure that it isn't exceeded. The systems engineers (*Engineer A*) pushes the data from their MBSE model to the ASOT (Nucleus) and informs *Engineer B* that data is available for a performance evaluation.

Engineer B, the conceptual aircraft designer, now pulls down the data that *Engineer A* had just uploaded by running ADAPT with the respective plugin blocks included into the workflow. ADAPT automatically downloads the data, runs the specified workflow (in this case an evaluation of the required take-off distance), and then uploads the ADAPT results to Nucleus. Now, *Engineer B* informs their counterpart (*Engineer A*) that the evaluation has completed.

Engineer A downloads the results directly into Cameo. They now inspect the take-off requirement and observe that it is violated. As a result, the systems engineer modifies the wing span of the vehicle, pushes this data back to Nucleus, and informs their colleague. *Engineer B* runs through the same process as before to evaluate the take-off performance of the modified configuration and upload the results. Informed about updated results, *Engineer A* again downloads the data into their SysML diagrams, discovering that the take-off requirement is now satisfied.

This example was the first integrated MBSE-MDO/aircraft design demonstration conducted within the REACT project. It presented not only the inclusion of stakeholder information in the form of requirements, but significantly improved the turnaround time on feedback regarding fulfillment of requirements and associated design changes. As the REACT project and its framework evolved, this demonstration has been repeated in modified forms to include more features.

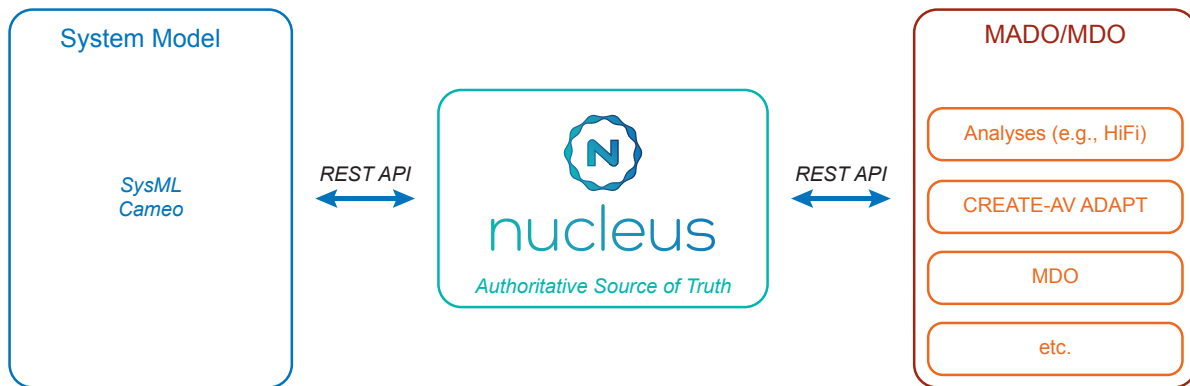
V. Ramifications

Chasing digital transformation, organizations seek to integrate MBSE with established design processes and connect to the Digital Thread. For example, AFRL/RQ's Digital Engineering strategy presented by Aielli [69] (Figure 13) illustrates a tight integration between stakeholders (left), while utilizing MBSE to enable an agile process and integrate the Digital Thread to increase traceability. However, this constitutes a seismic shift away from traditional processes, carrying widely-felt ramifications are requiring enabling technologies to achieve.

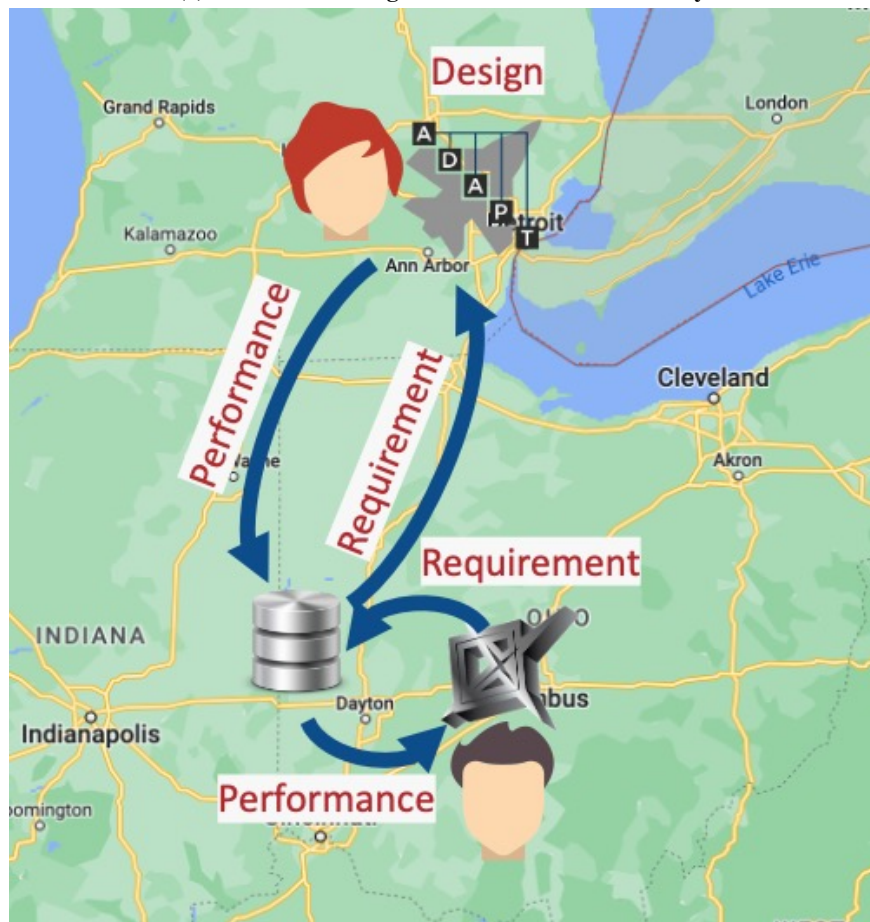
The tools developed by the REACT team, some of which are presented within this paper, are key enablers for this Digital Engineering vision. Nucleus provides a centralized ASOT that analyses, designs, and other data producers/consumers can leverage to accelerate knowledge transfer and increase traceability of designs and decision making. Nucleus specifically targets early stage design processes and analyses, allowing the inspection of decisions during later design stages.

Nucleus is of particular use to create a Digital Thread of these early processes due to its use of cloud-native technologies. It utilizes tools maintained by the Cloud Native Computing Foundation (CNCF), including Kubernetes and Keycloak. This approach allows Nucleus to be widely deployed from small on-premises installs to large-scale cloud setups that can scale horizontally. Using a REST API enables wide access to a variety of users, independent of platform or programming language.

Several diverse engineering use-cases that are able to interface with Nucleus were presented. In some cases, the use of the centralized ASOT enabled a team of engineers to work collaboratively in a manner and speed that would not have been possible otherwise. An integrated workflow between MBSE and MDO/design processes illustrated the potential to include a variety of stakeholders, from systems engineers to conceptual aircraft designers, while enabling completely geographically distributed processes.



(a) Architectural diagram of the MBSE-MDO study.



(b) Setup of the 2022 REACT MBSE-MDO/aircraft design demonstration

Fig. 12 Architecture and setup for the MBSE-MDO/aircraft design demonstration

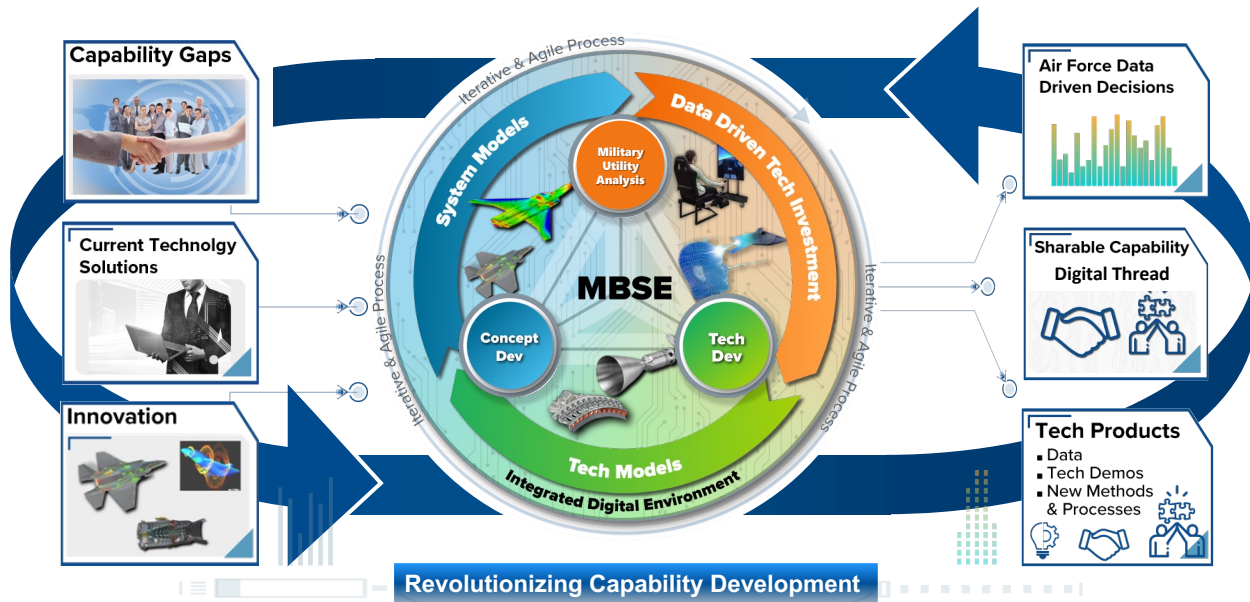


Fig. 13 AFRL/RQ's Digital Engineering vision [69]. Reproduced with permission.

However, tool development is only a part of the Digital Transformation process. Historically, data has been siloed between organizations and often even within large organizations. Unifying data in one location takes time and may, in some cases, be met with stiff resistance. Organizational cultures, especially of large organizations, have grown around a siloed structure and changing an entrenched culture is often difficult. And while collaborative, distributed work is desirable, it presupposes that collaborative tools such as instant messaging are available and widely used. Tools such as Nucleus are only one part of the solution. As data becomes more important and organizations store and interact with larger quantities of it, they must adopt policies governing its curation, storage, and disposal.

A specific aspect of data that is often neglected is the concept of a data life cycle. Storing data in a database like Nucleus may be convenient to the user, but retaining stale data after projects have ended jeopardizes database performance and maintainability while driving up operational costs. As such, it is insufficient for organizations to merely adopt a data-centric approach to design without considering and regulating data life. Cold storage (on tapes, etc.), is less accessible, but cheaper, and reduces clutter in active databases. As projects age, they must be transitioned from databases to archives. These archives are still retrievable, although less practically than an operational database.

Increasing the amount of data and number of locations data is stored will also inevitably require additional infrastructure. Moreover, it will require different infrastructure than some organizations currently use. Databases may require parallel servers (redundancy, traffic management), the ability to scale horizontally quickly, and the ability to recover from faults and crashes. Most cloud environments are built on Kubernetes, which supports all of these. However, not every aerospace organization may have Kubernetes clusters readily available, especially to rank-and-file engineers.

With the proliferation of modern cloud environments within the engineering space, cybersecurity must also become an increasing focus. Distributed computing and increasingly connected teams also increase organizations potential exposure to vulnerabilities. As such, organizations must establish or evolve their cybersecurity posture to both enable Digital Engineering tools and create certification processes for them to prevent exploitation.

Finally, new data guidelines and new soft/hardware may prove moot without lasting organizational change. Many organizations were not originally designed for the workflows discussed in this paper and may struggle to support a larger Digital Engineering process without adapting. Transitioning from siloed data and disciplines to a more integrated approach requires time and deliberate change. Organizations will also need to invest as much in their employees (and their training) as their infrastructure needs.

VI. Concluding Remarks

This paper presented a theoretical foundation for including data in MDO, how it relates to MBSE-MDO integration, and the integration of conceptual and preliminary design workflows into the Digital Thread. A data-augmented N^2 was introduced to connect past MDO concepts with MBSE and account for data locality and architecture. The paper described the creation of a central database, that can store structured data, unstructured data, and metadata as an ASOT between engineering workflows. Ontology concepts of the ASOT were presented that fully and generally describe engineering design problems. Four vignettes were presented that illustrate the interactions of different design tools with the ASOT, indicating that the developed database is widely applicable. Finally, ramifications on data policies, infrastructure, and organizational culture were discussed with an eye toward enabling future Digital Engineering workflows.

Acknowledgments

The authors acknowledge Marco de Lannoy Kobayashi's early development of the Nucleus database (then known by another name) under Dr. Rich Snyder's project leadership during the initial phase of the REACT project. Mr. Kobayashi's implementation from 2021 until 2023 offered a significant stepping stone that was developed into the current Nucleus code. The authors also acknowledge A.J. Field's (NAVAIR and CREATE-AV) help to integrate CREATE-AV to Nucleus and participating in the initial MBSE-MADO integration demonstration together with Mr. Kobayashi and Dr. Snyder. The authors also acknowledge the work to Nucleus conducted by Hangar 18 and Crown Point Technologies, including code review, authentication, role-based access control implementations, and work toward an Authority to Operate (ATO).

References

- [1] Wu, S., Wang, G., Lu, J., Yan, Y., Gong, Y., Dong, M., and Kiritisis, D., "Digital thread in engineering: Concept, state of art, and enabling framework," *Advanced Engineering Informatics*, Vol. 65, 2025, p. 103258. <https://doi.org/10.1016/j.aei.2025.103258>.
- [2] Hamstra, J. W. (ed.), *The F-35 Lightning II: From Concept to Cockpit*, American Institute of Aeronautics and Astronautics, Inc., Reston, VA, 2019. <https://doi.org/10.2514/4.105678>.
- [3] Kraft, E., "HPCMP CREATE™-AV and the Air Force Digital Thread," *53rd AIAA Aerospace Sciences Meeting*, American Institute of Aeronautics and Astronautics, Kissimmee, Florida, 2015, pp. 1–13. <https://doi.org/10.2514/6.2015-0042>.
- [4] Kraft, E. M., "The Air Force Digital Thread/Digital Twin - Life Cycle Integration and Use of Computational and Experimental Knowledge," *54th AIAA Aerospace Sciences Meeting*, American Institute of Aeronautics and Astronautics, San Diego, California, USA, 2016, pp. 1–22. <https://doi.org/10.2514/6.2016-0897>.
- [5] Kraft, E. M., "Developing a Digital Thread / Digital Twin Aerodynamic Performance Authoritative Truth Source," *2018 Aviation Technology, Integration, and Operations Conference*, American Institute of Aeronautics and Astronautics, Atlanta, Georgia, 2018, pp. 1–12. <https://doi.org/10.2514/6.2018-4003>.
- [6] Kraft, E. M., "Decision Analytics in a Lifecycle Digital Engineering Environment," *AIAA Scitech 2019 Forum*, American Institute of Aeronautics and Astronautics, San Diego, California, 2019, pp. 1–24. <https://doi.org/10.2514/6.2019-1364>.
- [7] Kraft, E. M., "Digital Engineering Enabled Systems Engineering Performance Measures," *AIAA Scitech 2020 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2020, pp. 1–24. <https://doi.org/10.2514/6.2020-0552>.
- [8] Kraft, E. M., "Transforming Ground and Flight Testing through Digital Engineering," *AIAA Scitech 2020 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2020, pp. 1–17. <https://doi.org/10.2514/6.2020-1840>.
- [9] Kraft, E. M., "A Knowledge-Graph Approach to Digital Materiel Management," *AIAA SCITECH 2025 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2025, pp. 1–13. <https://doi.org/10.2514/6.2025-1285>.
- [10] Kobryn, P., Tuegel, E. J., Zweber, J. V., and Kolonay, R. M., "Digital Thread and Twin for Systems Engineering: Design to Retirement," *55th AIAA Aerospace Sciences Meeting*, American Institute of Aeronautics and Astronautics, Grapevine, Texas, 2017, pp. 1–13. <https://doi.org/10.2514/6.2017-0876>.
- [11] Gharbi, A., Sarojini, D., Kallou, E., Harper, D. J., Petitgenet, V., Rancourt, D., Briceno, S. I., and Mavris, D. N., "A Single Digital Thread Approach to Aircraft Detailed Design," *55th AIAA Aerospace Sciences Meeting*, American Institute of Aeronautics and Astronautics, Grapevine, Texas, 2017, pp. 1–13. <https://doi.org/10.2514/6.2017-0693>.

- [12] Singh, V., and Willcox, K. E., "Engineering Design with Digital Thread," *AIAA Journal*, Vol. 56, No. 11, 2018, pp. 4515–4528. <https://doi.org/10.2514/1.J057255>.
- [13] Raghunath, C., Watson, L. T., Jrad, M., Kapania, R. K., and Kolonay, R. M., "Service ORiented Computing EnviRonment (SORCER) for deterministic global and stochastic aircraft design optimization: part 1," *Advances in aircraft and spacecraft science*, Vol. 4, No. 3, 2017, pp. 297–316. <https://doi.org/10.12989/AAS.2017.4.3.297>.
- [14] Kolonay, R. M., "MSTC Engineering – A Distributed and Adaptive Collaborative Design Computational Environment for Military Aerospace Vehicle Development and Technology Assessment," *AIAA Aviation 2019 Forum*, American Institute of Aeronautics and Astronautics, Dallas, Texas, 2019, pp. 1–16. <https://doi.org/10.2514/6.2019-2992>.
- [15] Kolonay, R., and Burton, S., "Object Models for Distributed Multidisciplinary Analysis and Optimization (MAO) Environments that Promotes CAE Interoperability," *10th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Albany, New York, 2004, pp. 1–16. <https://doi.org/10.2514/6.2004-4599>.
- [16] Kolonay, R., Thompson, E., Camberos, J., and Eastep, F., "Active Control of Transpiration Boundary Conditions for Drag Minimization with an Euler CFD Solver," *48th AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, American Institute of Aeronautics and Astronautics, Honolulu, Hawaii, 2007, pp. 1–14. <https://doi.org/10.2514/6.2007-1891>.
- [17] Bryson, D., Stanford, B., McClung, A., Sims, T., Miranda, J., Beran, P., and Parker, G., "Multidisciplinary Optimization of a Hovering Wing with a Service-Oriented Framework and Experimental Model Validation," *49th AIAA Aerospace Sciences Meeting including the New Horizons Forum and Aerospace Exposition*, American Institute of Aeronautics and Astronautics, Orlando, Florida, 2011, pp. 1–23. <https://doi.org/10.2514/6.2011-1133>.
- [18] Alyanak, E., Kolonay, R., Flick, P., Lindsley, N., and Burton, S., "Efficient Supersonic Air Vehicle Preliminary Conceptual Multi-Disciplinary Design Optimization Results," *12th AIAA Aviation Technology, Integration, and Operations (ATIO) Conference and 14th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Indianapolis, Indiana, 2012, pp. 1–19. <https://doi.org/10.2514/6.2012-5518>.
- [19] Burton, S., Alyanak, E., and Kolonay, R., "Efficient Supersonic Air Vehicle Analysis and Optimization Implementation using SORCER," *12th AIAA Aviation Technology, Integration, and Operations (ATIO) Conference and 14th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Indianapolis, Indiana, 2012, pp. 1–12. <https://doi.org/10.2514/6.2012-5520>.
- [20] Thompson, E., "Incorporating Nonlinear Aerodynamic Methods Into Transonic Preliminary Design Of A Wing," *12th AIAA Aviation Technology, Integration, and Operations (ATIO) Conference and 14th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Indianapolis, Indiana, 2012, pp. 1–29. <https://doi.org/10.2514/6.2012-5607>.
- [21] Allison, D. L., and Alyanak, E. J., "Development of Installed Propulsion Performance Model for High-Performance Aircraft Conceptual Design," *15th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Atlanta, GA, 2014, pp. 1–20. <https://doi.org/10.2514/6.2014-2725>.
- [22] Burton, S. A., Kolonay, R. M., and Sobolewski, M., "Multidisciplinary Analysis with SORCER using Domain-Specific Objects," *15th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Atlanta, GA, 2014, pp. 1–18. <https://doi.org/10.2514/6.2014-2180>.
- [23] Burton, S. A., Kao, J. Y., White, T. L., Reich, G. W., and Kolonay, R. M., "Air-Racer Design Exploration using Latin Hypercube and Genetic Algorithm Methods," *AIAA Scitech 2020 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2020, pp. 1–12. <https://doi.org/10.2514/6.2020-1392>.
- [24] Tidball, M., Beblo, R., Cruz-Gonzalez, T., and Frank, G., "Optimization of an Actuation System for Articulated Missiles," *AIAA SCITECH 2022 Forum*, American Institute of Aeronautics and Astronautics, San Diego, CA & Virtual, 2022, pp. 1–18. <https://doi.org/10.2514/6.2022-0143>.
- [25] Snyder, R. D., "A Cross-Language Remote Procedure Call Framework," *18th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Denver, Colorado, 2017, pp. 1–17. <https://doi.org/10.2514/6.2017-3822>.
- [26] Lupp, C. A., and Xu, A., "Creating a Universal Communication Standard to Enable Heterogeneous Multidisciplinary Design Optimization," *AIAA SCITECH 2024 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2024, pp. 1–22. <https://doi.org/10.2514/6.2024-1799>.

- [27] Google, “google Remote Procedure Call (gRPC),” , 2016. URL <https://grpc.io>.
- [28] Reuter, R. A., Iden, S., Snyder, R. D., and Allison, D. L., “An Overview of the Optimized Integrated Multidisciplinary Systems Program,” *57th AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, American Institute of Aeronautics and Astronautics, San Diego, California, USA, 2016, pp. 1–11. <https://doi.org/10.2514/6.2016-0674>.
- [29] Allison, D. L., and Kolonay, R. M., “Expanded MDO for Effectiveness Based Design Technologies: EXPEDITE Program Introduction,” *2018 Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Atlanta, Georgia, 2018, pp. 1–11. <https://doi.org/10.2514/6.2018-3419>.
- [30] Torres, F., and McCarthy, K., “Lockheed Martin Overview of the AFRL EXPEDITE Program: Power and Thermal Management System,” *AIAA Scitech 2020 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2020, pp. 1–34. <https://doi.org/10.2514/6.2020-1129>.
- [31] Lefebvre, T., Bartoli, N., Dubreuil, S., Panzeri, M., Lombardi, R., D’Ippolito, R., Della Vecchia, P., Nicolosi, F., and Ciampa, P. D., “Methodological enhancements in MDO process investigated in the AGILE European project,” *18th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Denver, Colorado, 2017, pp. 1–23. <https://doi.org/10.2514/6.2017-4140>.
- [32] van Gent, I., Ciampa, P. D., Aigner, B., Jepsen, J., La Rocca, G., and Schut, J., “Knowledge architecture supporting collaborative MDO in the AGILE paradigm,” *18th AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Denver, Colorado, 2017, pp. 1–19. <https://doi.org/10.2514/6.2017-4139>.
- [33] van Gent, I., Aigner, B., Beijer, B., and La Rocca, G., “A Critical Look at Design Automation Solutions for Collaborative MDO in the AGILE Paradigm,” *2018 Multidisciplinary Analysis and Optimization Conference*, American Institute of Aeronautics and Astronautics, Atlanta, Georgia, 2018, pp. 1–23. <https://doi.org/10.2514/6.2018-3251>.
- [34] ESTECO SpA, “VOLTA,” , 2023. URL <https://www.esteco.com/volta>, process Automation and Design Space Exploration Platform.
- [35] Department of Defense, “MIL-STD-881E: Work Breakdown Structures for Defense Materiel Items,” Military Standard MIL-STD-881E, United States Department of Defense, Oct. 2018. URL <https://www.acq.osd.mil/evm/resources/references.html>.
- [36] Field, A. J., Lickenbrock, M., McGough, W., and Syfrett, P., “An Overview of HPCMP CREATE-AV™ ADAPT,” *AIAA SCITECH 2022 Forum*, American Institute of Aeronautics and Astronautics, San Diego, CA & Virtual, 2022, pp. 1–16. <https://doi.org/10.2514/6.2022-1317>.
- [37] Field, A. J., Lickenbrock, M., Maclean, J., McGough, W., Syfrett, P., and Zuber, W., “Advancements in HPCMP CREATE-AV™ ADAPT as an MDAO Front-End,” *AIAA SCITECH 2023 Forum*, American Institute of Aeronautics and Astronautics, National Harbor, MD & Online, 2023, pp. 1–26. <https://doi.org/10.2514/6.2023-0043>.
- [38] Kao, J., “REACT Overview,” Presentation at AVTech Symposium, 2023.
- [39] Boggero, L., Ciampa, P. D., and Nagel, B., “The Application of the AGILE 4.0 MBSE Architectural Framework for the Modeling of System Stakeholders, Needs and Requirements,” *International Council of Aeronautical Sciences (ICAS) 2021*, Shanghai, 2021, pp. 1–13.
- [40] Ciampa, P. D., and Nagel, B., “Accelerating the Development of Complex Systems in Aeronautics via MBSE and MDAO: a Roadmap to Agility,” *AIAA AVIATION 2021 FORUM*, American Institute of Aeronautics and Astronautics, VIRTUAL EVENT, 2021, pp. 1–14. <https://doi.org/10.2514/6.2021-3056>.
- [41] Bussemaker, J., and Boggero, L., “From System Architecting to System Design and Optimization: A Link Between MBSE and MDAO,” *32nd Annual INCOSE International Symposium*, INCOSE, Detroit, MI, 2022, pp. 1–17.
- [42] Boggero, L., Lefebvre, T., Vankan, J., Beijer, B., and Saluzzi, V., “The AGILE 4.0 MBSE-MDAO Development Framework: Overview and Assessment,” *International Council of Aeronautical Sciences (ICAS) 2022*, Stockholm, 2022, pp. 1–11.
- [43] Jeyaraj, A. K., Tabesh, N., and Liscouet-Hanke, S., “Connecting Model-based Systems Engineering and Multidisciplinary Design Analysis and Optimization for Aircraft Systems Architecting,” *AIAA AVIATION 2021 FORUM*, American Institute of Aeronautics and Astronautics, VIRTUAL EVENT, 2021, pp. 1–17. <https://doi.org/10.2514/6.2021-3077>.
- [44] Bussemaker, J. H., De Smedt, T., La Rocca, G., Ciampa, P. D., and Nagel, B., “System Architecture Optimization: An Open Source Multidisciplinary Aircraft Jet Engine Architecting Problem,” *AIAA AVIATION 2021 FORUM*, American Institute of Aeronautics and Astronautics, VIRTUAL EVENT, 2021, pp. 1–20. <https://doi.org/10.2514/6.2021-3078>.

- [45] Jeyaraj, A. K., Bussemaker, J. H., Liscouët-Hanke, S., and Boggero, L., "Systems architecting: a practical example of design space modeling and safety-based filtering within the AGILE 4.0 project," *International Council of Aeronautical Sciences (ICAS) 2022*, Stockholm, 2022, pp. 1–16.
- [46] Bussemaker, J., Sánchez, R. G., Fouda, M., Boggero, L., and Nagel, B., "Function-Based Architecture Optimization: An Application to Hybrid-Electric Propulsion Systems," *INCOSE International Symposium*, Vol. 33, No. 1, 2023, pp. 251–272. <https://doi.org/10.1002/iis2.13020>.
- [47] Cabaleiro, C., Fioriti, M., Boggero, L., Corpino, S., and Ciampa, P. D., "Assessment of New Technologies in a Multi-Disciplinary Design Analysis and Optimization Environment Including RAMS and Cost Disciplines," *International Council of Aeronautical Sciences (ICAS) 2021*, Shanghai, 2021, pp. 1–14.
- [48] Mandorino, M., Della Vecchia, P., Corcione, S., Nicolosi, F., Trifari, V., Cerino, G., Fioriti, M., Cabaleiro De La Hoz, C., Lefebvre, T., Charbonnier, D., Wang, Z., and Peeters, D., "Multidisciplinary Design and Optimization of Regional Jet Retrofitting Activity," *AIAA AVIATION 2022 Forum*, American Institute of Aeronautics and Astronautics, Chicago, IL & Virtual, 2022, pp. 1–14. <https://doi.org/10.2514/6.2022-3933>.
- [49] Fioriti, M., Cabaleiro, C., Lefebvre, T., Vecchia, P. D., Mandorino, M., Liscouët-Hanke, S., and Jeyaraj, A., "Certification driven design from stakeholders' needs to MDAO formulation within AGILE 4.0 project," *International Council of Aeronautical Sciences (ICAS) 2022*, Stockholm, 2022, pp. 1–14.
- [50] Saves, P., Bartoli, N., Diouane, Y., Lefebvre, T., Morlier, J., David, C., Nguyen Van, E., and Defoort, S., "Multidisciplinary design optimization with mixed categorical variables for aircraft design," *AIAA SCITECH 2022 Forum*, American Institute of Aeronautics and Astronautics, San Diego, CA & Virtual, 2022, pp. 1–22. <https://doi.org/10.2514/6.2022-0082>.
- [51] Allison, D. L., West, S., McCarthy, T. R., and Cribb, M. W., "An Approach to Implement Model Based Engineering in an Industry Setting," *AIAA Scitech 2021 Forum*, American Institute of Aeronautics and Astronautics, VIRTUAL EVENT, 2021, pp. 1–13. <https://doi.org/10.2514/6.2021-0237>.
- [52] Allison, D. L., Cribb, M. W., McCarthy, T., and LaRowe, R., "Authoritative Sources of Truth and Consistency in Digital Engineering," *AIAA SCITECH 2023 Forum*, American Institute of Aeronautics and Astronautics, National Harbor, MD & Online, 2023, pp. 1–11. <https://doi.org/10.2514/6.2023-1404>.
- [53] Roohi, Z. A., Abelezele, S., Fronk, G., Al Fawares, R., Pinon-Fischer, O. J., Gharbi, A., Mavris, D. N., Oroz, J., Petersen, M., Karl, A., Matlik, J. F., and Schwering, B., "Implementing the Digital Thread - A Proof-of-Concept," *AIAA SCITECH 2023 Forum*, American Institute of Aeronautics and Astronautics, National Harbor, MD & Online, 2023, pp. 1–24. <https://doi.org/10.2514/6.2023-1405>.
- [54] Fazal, B., Schmidt, J., Phillips, B. D., Ordaz, I., and Moore, K., "Integration of Uncertainty Quantification in a Model-Based Systems Analysis and Engineering Framework," *AIAA AVIATION FORUM AND ASCEND 2024*, American Institute of Aeronautics and Astronautics, Las Vegas, Nevada, 2024, pp. 1–7. <https://doi.org/10.2514/6.2024-4559>.
- [55] Carrere, A., Wakayama, S., Engelbeck, R., Shi, M., Arias, Y., and Gablonsky, J., "Establishing Next Generation Collaboration in Model Based Systems Analysis & Engineering," *AIAA SCITECH 2026 Forum*, AIAA, Orlando, FL, 2026, pp. 1–20.
- [56] Engelbeck, R., Shi, M., Gablonsky, J., Ocampo, E., Wakayama, S., and Carrere, A., "The Aircraft Data Hierarchy: A Modern Aircraft Configuration Data Standard," *AIAA SCITECH 2026 Forum*, AIAA, Orlando, FL, 2026, pp. 1–26.
- [57] Gablonsky, J., Carrere, A., Engelbeck, R., Goldade, M., Musiak, J., Ocampo, E., and Wakayama, S., "Model-based Systems Analysis & Engineering: Standard Evaluator," *AIAA SCITECH 2026 Forum*, AIAA, Orlando, FL, 2026, pp. 1–15.
- [58] Mokotoff, P. R., Shi, M., Carrere, A., and Cinar, G., "Weakly Coupling MBSE and MDAO via a Tool-Neutral Authoritative Source of Truth," *AIAA SCITECH 2026 Forum*, AIAA, Orlando, FL, 2026, pp. 1–15.
- [59] Arias, Y., and Wood, D., "Model-based Systems Analysis & Engineering: Comprehensive and Narrow Model Sharing and Collaboration Approaches," *AIAA SCITECH 2026 Forum*, AIAA, Orlando, FL, 2026, pp. 1–19.
- [60] Wakayama, S., Plybon, R. C., and Carrere, A., "Aircraft Model for Model-Based Systems Analysis & Engineering," *AIAA SCITECH 2026 Forum*, AIAA, Orlando, FL, 2026, pp. 1–16.
- [61] Lupp, C. A., Deaton, J. D., Kao, J. Y., Clark, D. L., Smith, H. A., and Xu, A., "The Monolith Must Die: Why the Correct Partitioning of MDO Problems is Necessary to Enable Large-Scale Applications," *AIAA SCITECH 2025 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2025, pp. 1–. <https://doi.org/10.2514/6.2025-0579>.

- [62] Gray, J. S., Hwang, J. T., Martins, J. R. R. A., Moore, K. T., and Naylor, B. A., “OpenMDAO: an open-source framework for multidisciplinary design, analysis, and optimization,” *Structural and Multidisciplinary Optimization*, Vol. 59, No. 4, 2019, pp. 1075–1104. <https://doi.org/10.1007/s00158-019-02211-z>.
- [63] Jasa, J. P., Hwang, J. T., and Martins, J. R. R. A., “Open-source coupled aerostructural optimization using Python,” *Structural and Multidisciplinary Optimization*, Vol. 57, No. 4, 2018, pp. 1815–1827. <https://doi.org/10.1007/s00158-018-1912-8>.
- [64] Raymer, D., *Aircraft design: A conceptual approach*, AIAA education series, American Institute of Aeronautics and Astronautics, Incorporated, 2018.
- [65] Torenbeek, E., *Synthesis of Subsonic Airplane Design*, 1st ed., Springer, Dordrecht, 1982.
- [66] Novotny, N. L., Sandler, D. A., and Wukie, N. A., “Gradient-based aeroelastic optimization of a generic flying wing model,” *AIAA SCITECH 2025 Forum*, AIAA, Orlando, FL, 2025, p. 1749.
- [67] Sandler, D. A., and Novotny, N. L., “Aeroelastic optimization benchmark investigations using ESP, FUN3D, TACS, and FUNtoFEM,” *AIAA SCITECH 2026 Forum*, AIAA, Orlando, FL, 2026, pp. 1–15.
- [68] Gray, A. C., and Martins, J. R., “A Proposed Benchmark Model for Practical Aeroelastic Optimization of Aircraft Wings,” *AIAA SCITECH 2024 Forum*, American Institute of Aeronautics and Astronautics, Orlando, FL, 2024, pp. 1–24. <https://doi.org/10.2514/6.2024-2775>.
- [69] Aielli, C., “RQ Digital Engineering,” , 2025. Published: Presentation.